Test Document

for

UniPool

Version 1.0

Prepared by

Group #16 **Group Name: Undefined_Behaviour**

Aalok	240011	aalok24@iitk.ac.in
Alankrit Gupta	220110	alankritg22@iitk.ac.in
Anubhav Rana	210162	anubhavr21@iitk.ac.in
Aryan Nigam	220227	naryan22@iitk.ac.in
Bhavishya Gupta	220295	bhavishyap22@iitk.ac.in
Gagandeep Kolambe	220393	gagandeep22@iitk.ac.in
Harshit Gupta	220439	gharshit22@iitk.ac.in
Karan Kumar	230533	karanm23@iitk.ac.in
Naman Mohan Singh	230678	mohans23@iitk.ac.in
Pavi Agarwal	220762	apavi22@iitk.ac.in

Course: CS253

Mentor TA: V Arvind

Date: 3rd April 2026

Contents

CONTENTS II

REVISIONS III

1 INTRODUCTION 1

1.1 TEST STRATEGY 1
1.2 TESTING TIMELINE 1
1.3 TESTERS 1
1.4 COVERAGE CRITERIA 1
1.5 TOOLS USED FOR TESTING 1

2 UNIT TESTING 3

2.1 AUTHENTICATION 3
2.2 PROFILE MANAGEMENT 14
2.3 RIDE HANDLING 18
2.4 CHAT/MESSAGING 27

3 INTEGRATION TESTING 32
3.1 AUTHENTICATION 32
3.2 CREATE RIDE POST 38
3.3 FIND AND FILTER RIDES 42
3.4 EDIT PROFILE 47
3.5 CHAT SYSTEM 50
3.6 MY ACTIVITY 55
4 SYSTEM TESTING 65
5 CONCLUSION 69

APPENDIX A – GROUP LOG 71

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Aalok Alankrit Gupta Anubhav Rana Aryan Nigam Bhavishya Gupta Gagandeep Kolambe Harshit Gupta Karan Kumar Naman Mohan Singh Pavi Agarwal	Preliminary draft	28/03/25

1 INTRODUCTION

1.1 Test Strategy

We performed manual integration testing to validate the end-to-end interaction between the Flutter frontend and the Firebase backend. For unit testing of the frontend, we used the flutter_test framework. For backend-facing unit testing of Firestore write logic and Authentication flows, we used the fake_cloud_firestore and firebase_auth_mocks Dart packages, which allow Firebase SDK calls to be exercised against an in-memory database without connecting to a live Firebase project. For testing Firestore Security Rules we used the Firebase Local Emulator Suite (firebase emulators:start) in conjunction with the @firebase/rules-unit-testing JavaScript library.

1.2 Testing Timeline

Unit testing of authentication and Firestore interaction logic was conducted in parallel with the implementation phase. Integration testing and system testing were performed after the core implementation phase was complete, approximately one week before the submission deadline.

1.3 Testers

Testing was distributed across all group members. Unit test scripts were primarily authored by members familiar with the Dart/Flutter testing framework. Integration and system tests were conducted collaboratively during dedicated group sessions, with each member responsible for testing features they did not personally develop, in order to reduce confirmation bias.

1.4 Coverage Criteria

We ensured that testing achieves branch coverage, statement coverage, and function coverage for all critical frontend validation logic. For Firestore-facing operations, we verified that each code path that writes to or reads from Firestore produces the correct document structure. Security rule correctness was verified by exercising allowed and denied access patterns. Frontend widget logic was tested to confirm that the correct error alerts are displayed for all invalid input states.

1.5 Tools Used for Testing

The following tools were used during the testing process:

- **flutter_test:** The standard Dart/Flutter unit and widget testing framework. Used to write automated tests for validation logic, Firestore document construction, and widget rendering.
- **fake_cloud_firestore (pub.dev):** An in-memory Firestore emulator for Dart. Used to test Firestore read and write operations without connecting to a live Firebase project, enabling fast, deterministic unit tests.

- **firebase_auth_mocks (pub.dev):** A mock implementation of Firebase Authentication for Dart. Used to simulate signed-in and signed-out user states without requiring real Firebase credentials.
- **Firebase Local Emulator Suite:** Google's official local emulation environment. Used to test Firestore Security Rules by running firebase emulators:start and executing @firebase/rules-unit-testing scripts against the local Firestore instance.
- **Dart DevTools / Flutter Inspector:** Used for debugging the widget tree during UI development and for inspecting state during manual integration testing.
- **Manual Testing:** Full user-flow walkthroughs on Android emulators and physical devices were conducted for all integration and system test scenarios.

2 UNIT TESTING

The Flutter frontend and its Firestore interaction logic were treated as separate units during this phase. Firestore-facing unit tests use fake_cloud_firestore to intercept all SDK calls so that no real database is contacted. Authentication tests use firebase_auth_mocks. Frontend widget validation logic is tested using flutter_test's widget testing capabilities.

We conducted thorough unit testing for each feature section. Each subsection marked .a documents a Firestore/backend-facing test with a Dart code snippet; each subsection marked .b documents a frontend widget/UI validation test with a screenshot placeholder.

2.1 Authentication

2.1.1.a Sign In with Email and Password

Firebase SDK Path: FirebaseAuth.instance.signInWithEmailAndPassword()

Test Owner: Naman Mohan Singh

Test Description: Verify that the sign-in method succeeds when valid credentials are provided, that it throws a FirebaseAuthException for wrong credentials, and that the session is established correctly in the mock auth instance.

Test Results: The mock auth correctly returns a UserCredential on valid input and throws expected FirebaseAuthException codes (wrong-password, user-not-found) on invalid input.

```
// test/auth/sign_in_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:firebase_auth_mocks/firebase_auth_mocks.dart';

void main() {
  group('SignIn', () {

    test('valid credentials return UserCredential', () async {
      final mockUser = MockUser(uid: 'uid123', email: 'test@iitk.ac.in');
      final auth = MockFirebaseAuth(mockUser: mockUser);
      final cred = await auth.signInWithEmailAndPassword(
          email: 'test@iitk.ac.in',
          password: 'password123',
      );
      expect(cred.user?.uid, equals('uid123'));
    });

    test('wrong password throws FirebaseAuthException', () async {
      final auth = MockFirebaseAuth(authExceptions: {
        'signInWithEmailAndPassword':
            FirebaseAuthException(code: 'wrong-password'),
      });
      expect(
        () => auth.signInWithEmailAndPassword(
            email: 'test@iitk.ac.in', password: 'bad'),
        throwsA(isA<FirebaseAuthException>()),
      );
    });
  });
}
```

2.1.1.b Sign In — Frontend Validation

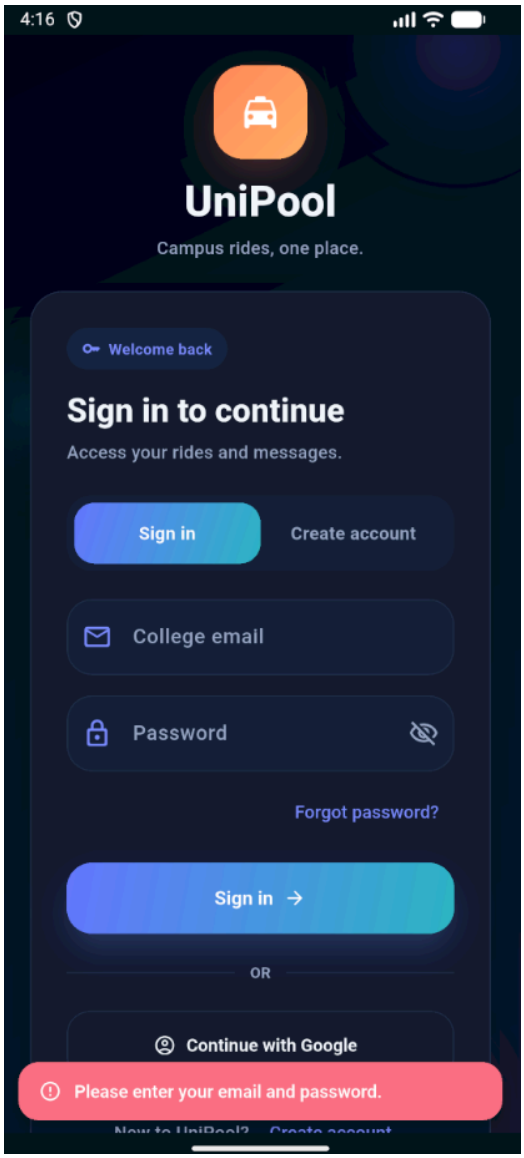
Test Owner: Karan Kumar

Test Date: [21/03/26]

Test Description: The sign-in screen's validation logic was exercised to verify that empty fields are caught before an API call is made and that the error snack bar is surfaced via showAppSnackBar. The loading flag correctly disables the button during the async operation.

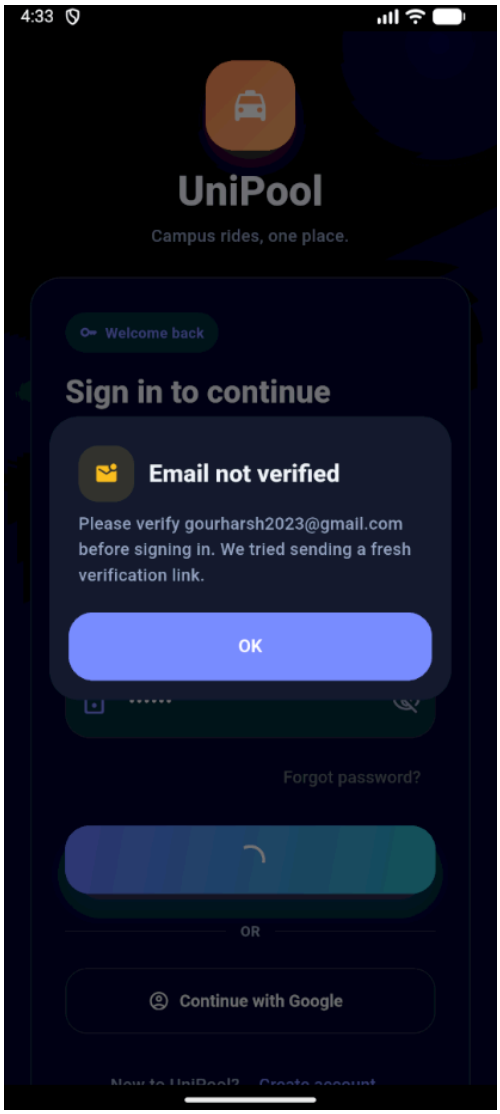
Test Results: Appropriate snack bar alerts appeared for empty fields and incorrect credentials. The loading indicator appeared and the button was disabled during the async call.

Check for empty email/password fields:



[Authentication Screen — Sign-in tapped with both fields empty; red snack bar at bottom reads "Please enter your email and password."]

Check for unverified email:



[Authentication Screen — Valid credentials entered but email not verified; user is signed out; snack bar reads "Verify your email before signing in. A fresh link has been sent."]

2.1.2.a Sign Up — Firestore Document Creation

Firebase SDK Path: FirebaseAuth.instance.createUserWithEmailAndPassword() + FirebaseFirestore.instance.collection('users').doc(uid).set({...})

Test Owner: Anubhav Rana

Test Description: Verify that account creation writes the correct document structure to the Firestore users/ collection, including the uid, email, name, ridesCompleted: 0, and createdAt fields.

Test Results: The Firestore users/{uid} document was created with all expected fields. ridesCompleted was initialised to 0.

```
// test/auth/sign_up_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:firebase_auth_mocks/firebase_auth_mocks.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

void main() {
  test('sign up creates Firestore user document', () async {
    final mockUser = MockUser(uid: 'uid456', email: 'new@iitk.ac.in');
    final auth = MockFirebaseAuth(mockUser: mockUser);
    final db = FakeFirebaseFirestore();

    await auth.createUserWithEmailAndPassword(
        email: 'new@iitk.ac.in', password: 'pass12345');

    // Simulate the Firestore write from AuthScreen.signUp()
    await db.collection('users').doc(mockUser.uid).set({
      'uid': mockUser.uid,
      'email': mockUser.email,
      'name': 'new',
      'ridesCompleted': 0,
      'createdAt': Timestamp.now(),
    });

    final snap = await db.collection('users').doc(mockUser.uid).get();
    expect(snap.exists, isTrue);
    expect(snap.get('ridesCompleted'), equals(0));
  });
}
```

2.1.2.b Sign Up — Frontend Validation

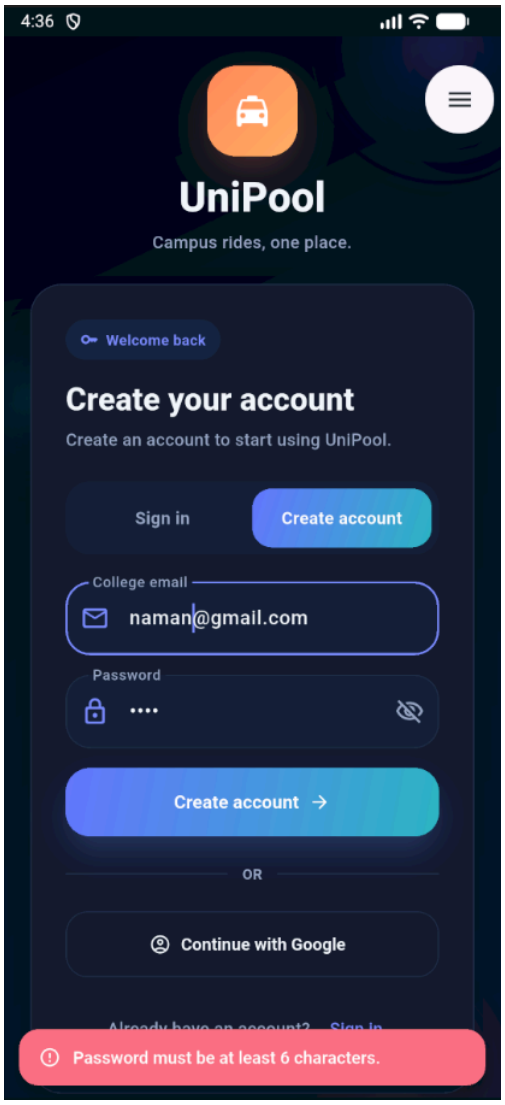
Test Owner: Pavi Agarwal

Test Date: [22/03/26]

Test Description: Verify that a password shorter than six characters is rejected before any API call is made, and that the empty-field guard triggers correctly.A verification link is sent.

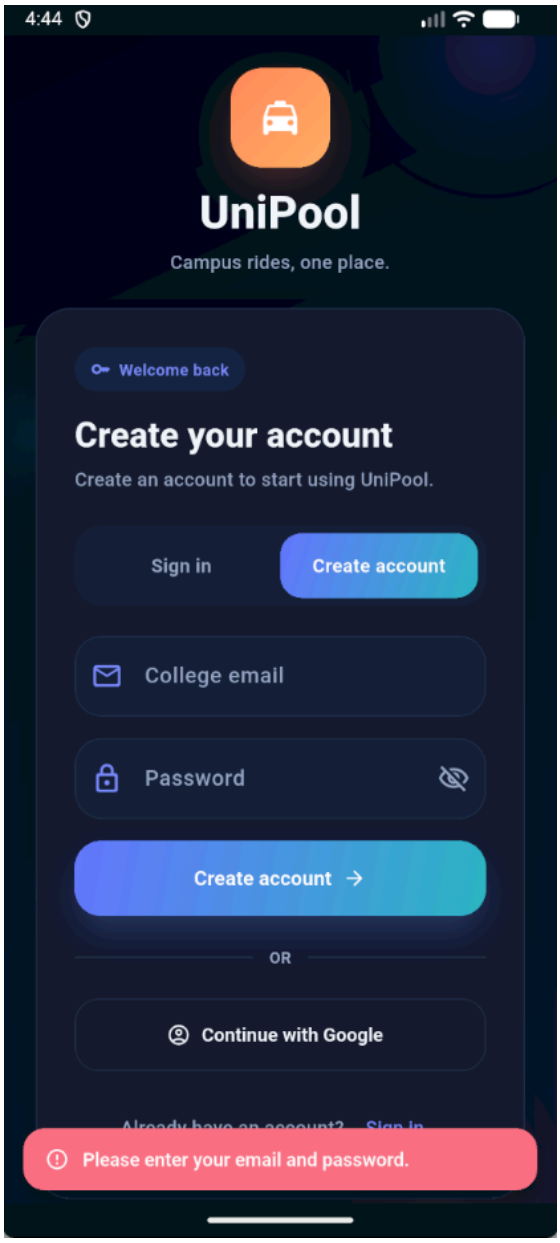
Test Results: Appropriate snack bars appeared for empty fields and short passwords. A verification confirmation snack bar appeared on successful registration. Verification email was sent.

Check for password shorter than 6 characters:



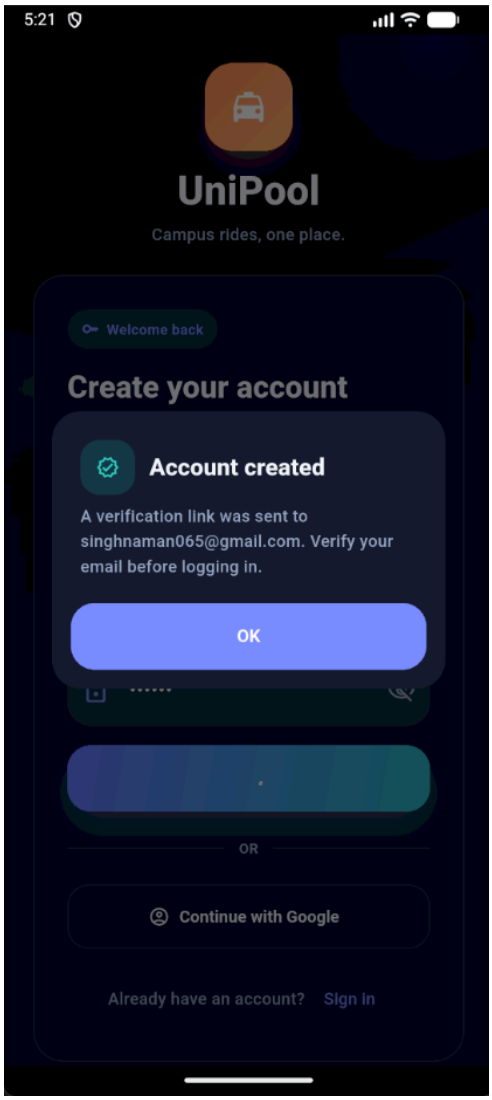
[Authentication Screen — Sign-up attempted with 4-character password; snack bar reads "Password must be at least 6 characters."]

Check for empty fields on sign-up:



[Authentication Screen — Sign-up attempted with both fields blank; snack bar reads "Please enter your email and password."]

Verification Email:



[Authentication Screen —"Verfication Email is Sent."]

2.1.3.a Password Reset Guard

Firebase SDK Path: FirebaseAuth.instance.sendPasswordResetEmail()

Test Owner: Alankrit Gupta

Test Description: Verify that sendPasswordResetEmail() is called only when a non-empty email is present, and that an empty email field triggers the error guard without any Firebase call.

Test Results: Empty email correctly prevented the Firebase call. A valid email triggered sendPasswordResetEmail() and a success snack bar was shown.

```
// test/auth/password_reset_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:firebase_auth_mocks/firebase_auth_mocks.dart';

void main() {
  group('PasswordReset', () {

    test('valid email dispatches reset link', () async {
      final auth = MockFirebaseAuth();
      // Should not throw
      await auth.sendPasswordResetEmail(email: 'test@iitk.ac.in');
    });

    test('empty email does not call sendPasswordResetEmail', () {
      const email = '';
      bool wasCalled = false;
      if (email.isNotEmpty) { wasCalled = true; }
      expect(wasCalled, isFalse);
    });
  });
}
```

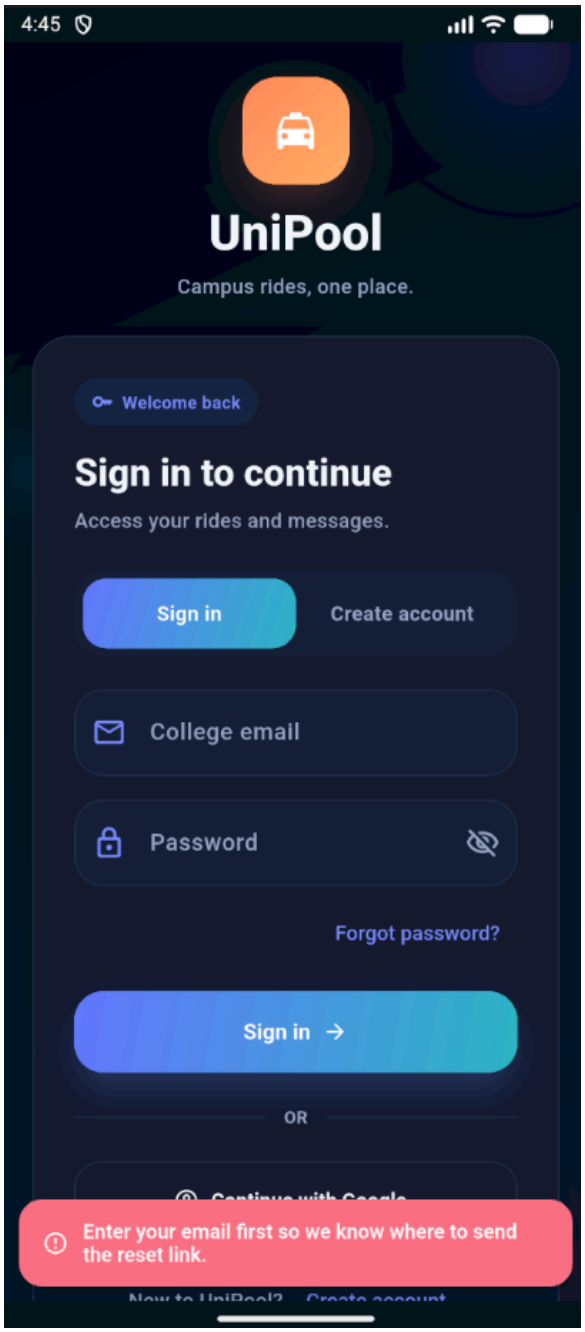
2.1.3.b Password Reset — Frontend Validation

Test Owner: Aalok

Test Date: [22/03/26]

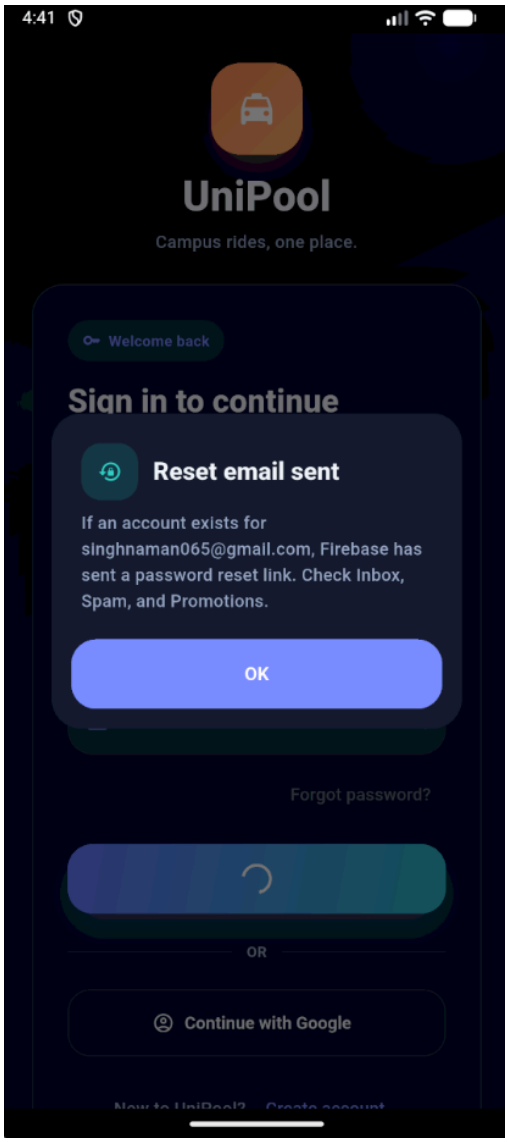
Test Results: The system correctly guarded the API call. Empty email produced the instructional error snack bar. Non-empty email produced the success snack bar.

Check for empty email before password reset:



[Authentication Screen — "Forgot password?" tapped with empty email field; snack bar reads "Enter your email first so we know where to send the reset link."]

Check for successful reset email dispatch:



[Authentication Screen — "Forgot password?" tapped with valid email; pop-up-bar reads "Password reset link sent. Check your inbox."]

2.1.4.a Google Sign-In — Firestore Profile Creation

Firebase SDK Path: _ensureGoogleProfile(): FirebaseFirestore.instance.collection('users').doc(uid).set({...})

Test Owner: Naman Mohan Singh

Test Description: Verify that on first Google sign-in, a Firestore user document is created for the UID with correct initial fields. On subsequent sign-ins, the existing document is not overwritten.

Test Results: First sign-in: document created with uid, email, name, photoUrl, ridesCompleted: 0, createdAt. Subsequent sign-in: existing document unchanged.

```
// test/auth/google_sign_in_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:firebase_auth_mocks/firebase_auth_mocks.dart';

Future<void> ensureGoogleProfile(dynamic user, dynamic db) async {
  final ref = db.collection('users').doc(user.uid);
  final snap = await ref.get();
  if (snap.exists) return;
  await ref.set({'uid': user.uid, 'ridesCompleted': 0});
}

void main() {
  test('creates document if not exists', () async {
    final db = FakeFirebaseFirestore();
    final user = MockUser(uid: 'google123');
    await ensureGoogleProfile(user, db);
    final snap = await db.collection('users').doc(user.uid).get();
    expect(snap.exists, isTrue);
  });

  test('does not overwrite existing document', () async {
    final db = FakeFirebaseFirestore();
    final user = MockUser(uid: 'google123');
    await db.collection('users').doc(user.uid)
        .set({'uid': user.uid, 'ridesCompleted': 7});
    await ensureGoogleProfile(user, db);
    final snap = await db.collection('users').doc(user.uid).get();
    expect(snap.get('ridesCompleted'), equals(7));
  });
}
```

2.2 Profile Management

2.2.1.a Update Display Name — Merge Write

Firebase SDK Path: FirebaseFirestore.instance.collection('users').doc(uid).set({'name': ...}, SetOptions(merge: true))

Test Owner: Naman Mohan Singh

Test Description: Verify that the _saveProfile() merge write updates only the name field without overwriting ridesCompleted or photoUrl.

Test Results: The merge write correctly updated name while leaving ridesCompleted unchanged at its existing value.

```
// test/profile/update_name_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

void main() {
  test('merge write preserves unrelated fields', () async {
    final db = FakeFirebaseFirestore();
    const uid = 'user001';

    // Seed existing document
    await db.collection('users').doc(uid).set({
      'uid': uid, 'name': 'OldName', 'ridesCompleted': 5,
    });

    // Simulate _saveProfile() with SetOptions(merge: true)
    await db.collection('users').doc(uid).set(
        {'name': 'NewName'}, SetOptions(merge: true));

    final snap = await db.collection('users').doc(uid).get();
    expect(snap.get('name'), equals('NewName'));
    expect(snap.get('ridesCompleted'), equals(5));
  });
}
```

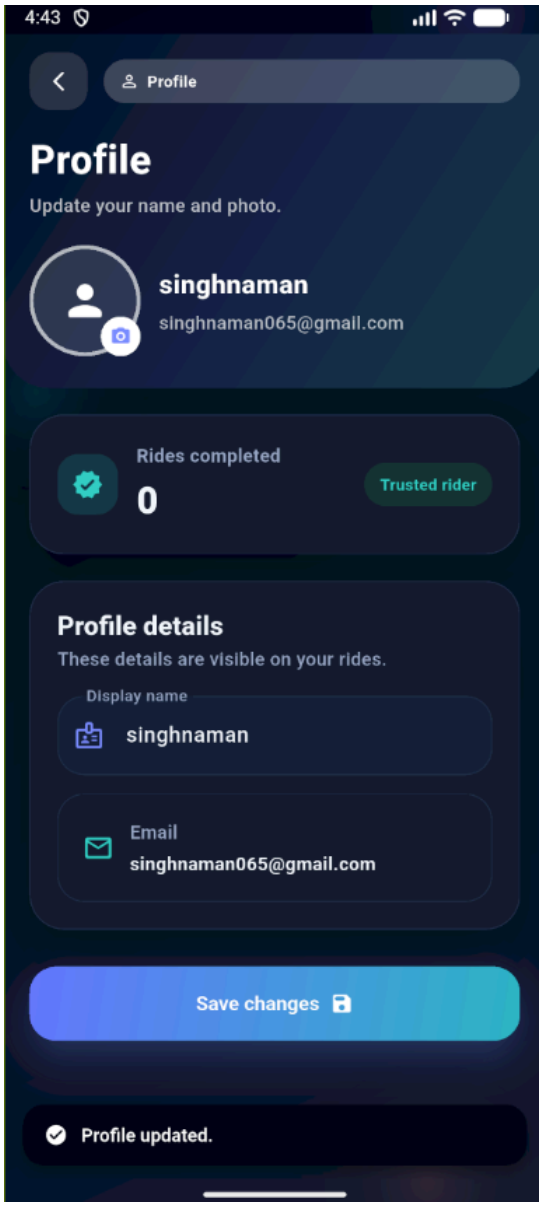
2.2.1.b Edit Profile — Frontend Validation

Test Owner: Bhavishya Gupta

Test Date: [22/03/26]

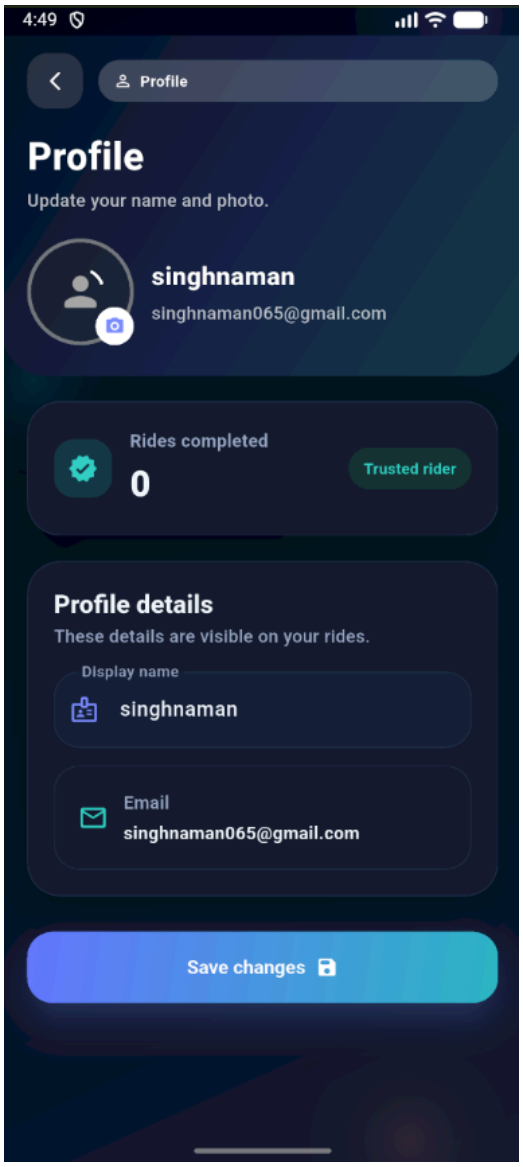
Test Results: Save Changes triggered a Firestore update with the new name. A success snack bar "Profile updated." appeared. The photo upload progress indicator appeared during upload.

Check for successful name save:



[Profile Screen — Updated display name entered; "Save Changes" tapped; snack bar reads "Profile updated."]

Check for photo upload progress indicator:(Paid Version Expired so will not store image)



[Profile Screen — Photo upload in progress; circular progress indicator overlaid on the avatar circle.]

2.2.2.a Rides Completed Counter Increment

Firebase SDK Path: FieldValue.increment(1) on users/{uid}.ridesCompleted

Test Owner: Aryan Nigam

Test Description: Verify that _completeRide() atomically increments ridesCompleted by 1 and sets ride status to "completed".

Test Results: ridesCompleted incremented from 3 to 4. Ride status changed from "open" to "completed".

```
// test/profile/rides_completed_test.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('marking complete increments ridesCompleted by 1', () async {
    final db = FakeFirebaseFirestore();
    await db.collection('rides').doc('ride01')
        .set({'status': 'open'});
    await db.collection('users').doc('leader01')
        .set({'ridesCompleted': 3});

    // Simulate _completeRide()
    await db.collection('rides').doc('ride01')
        .update({'status': 'completed'});
    await db.collection('users').doc('leader01')
        .update({'ridesCompleted': FieldValue.increment(1)});

    final user = await db.collection('users').doc('leader01').get();
    expect(user.get('ridesCompleted'), equals(4));
  });
}
```

2.3 Ride Handling

2.3.1.a Create Ride — Firestore Document Structure

Firebase SDK Path: FirebaseFirestore.instance.collection('rides').add({...})

Test Owner: Naman Mohan Singh

Test Description: Verify that _submitRide() writes a Firestore document with all required fields: source, destination, rideDate (ISO 8601), leaderId, leaderName, status ("open"), and participants (empty list). Also verify the source == destination guard.

Test Results: Document written with all expected fields. status was "open", participants was empty. Source-equals-destination guard prevented the write when fields were identical.

```
// test/rides/create_ride_test.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  group('CreateRide', () {

    test('write produces expected document structure', () async {
      final db = FakeFirebaseFirestore();
      const src = 'Hall 1', dst = 'Kanpur Central Railway Station';
      final date = DateTime(2026, 4, 15);

      await db.collection('rides').add({
        'source': src,
        'destination': dst,
        'rideDate': date.toIso8601String(),
        'leaderId': 'uid001',
        'leaderName': 'testuser',
        'status': 'open',
        'participants': [],
        'createdAt': Timestamp.now(),
      });

      final snaps = await db.collection('rides').get();
      expect(snaps.docs.length, equals(1));
      final data = snaps.docs.first.data();
      expect(data['source'], equals('Hall 1'));
      expect(data['status'], equals('open'));
      expect(data['participants'], isEmpty);
    });

    test('source == destination is rejected client-side', () {
      const src = 'Hall 1', dst = 'Hall 1';
      expect(src == dst, isTrue); // guard prevents Firestore write
    });
  });
}
```

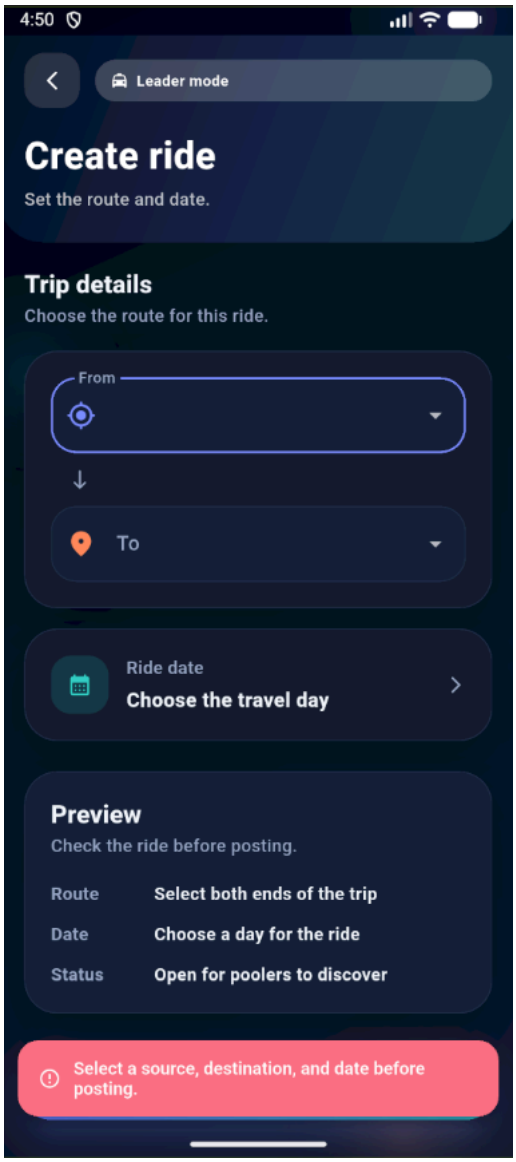
2.3.1.b Create Ride — Frontend Validation

Test Owner: Gagandeep Kolambe

Test Date: [24/03/26]

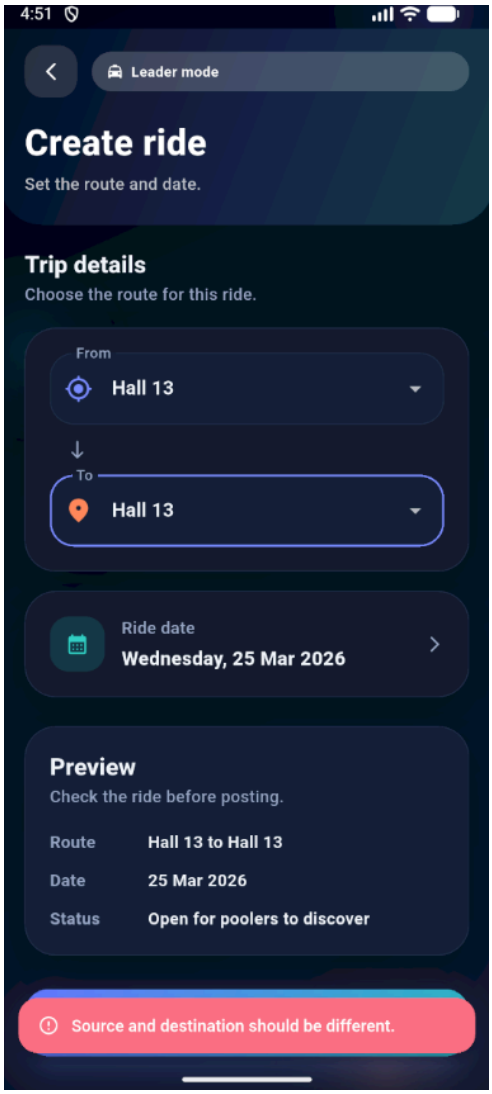
Test Results: Appropriate snack bars appeared for missing fields and identical source/destination. Post button showed loading indicator during write. Success snack bar appeared on valid submission.

Check for missing fields:



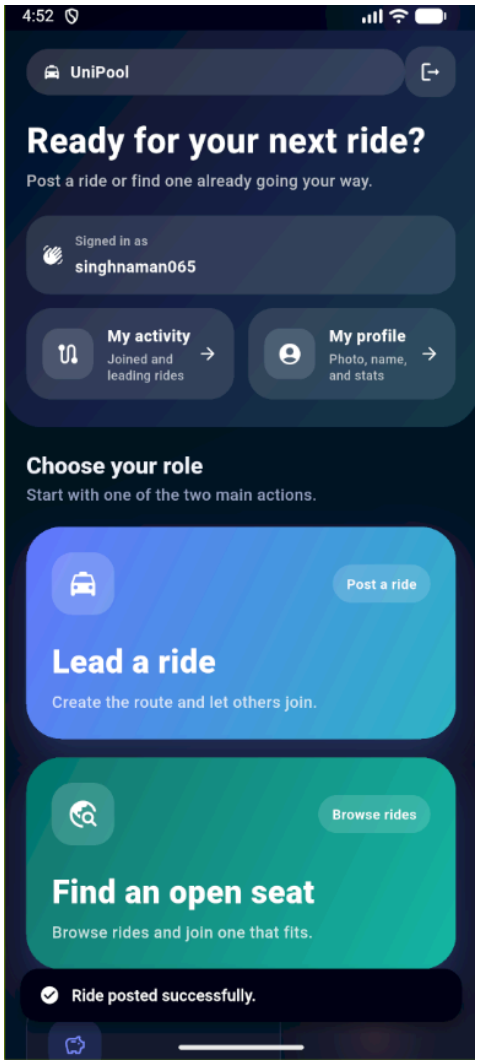
[Create Ride Screen — Post tapped with source and destination unselected; snack bar reads "Select a source, destination, and date before posting."]

Check for identical source and destination:



[Create Ride Screen — Same location selected for both source and destination; snack bar reads "Source and destination should be different."]

Check for successful ride post:



[Create Ride Screen — All fields populated; Post button shows loading indicator during write; after success user returns to HomeScreen with snack bar "Ride posted successfully."]

2.3.2.a Delete Ride

Firebase SDK Path: FirebaseFirestore.instance.collection('rides').doc(rideId).delete()

Test Owner: Harshit Gupta

Test Description: Verify that deleting a ride document removes it from Firestore entirely and that a subsequent .get() returns exists == false.

Test Results: Ride document was absent after delete. exists == false was confirmed.

```
// test/rides/delete_ride_test.dart
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('delete ride removes document from Firestore', () async {
    final db = FakeFirebaseFirestore();
    final ref = db.collection('rides').doc('rideXYZ');
    await ref.set({'source': 'Hall 1', 'status': 'open'});

    await ref.delete();
    final snap = await ref.get();
    expect(snap.exists, isFalse);
  });
}
```

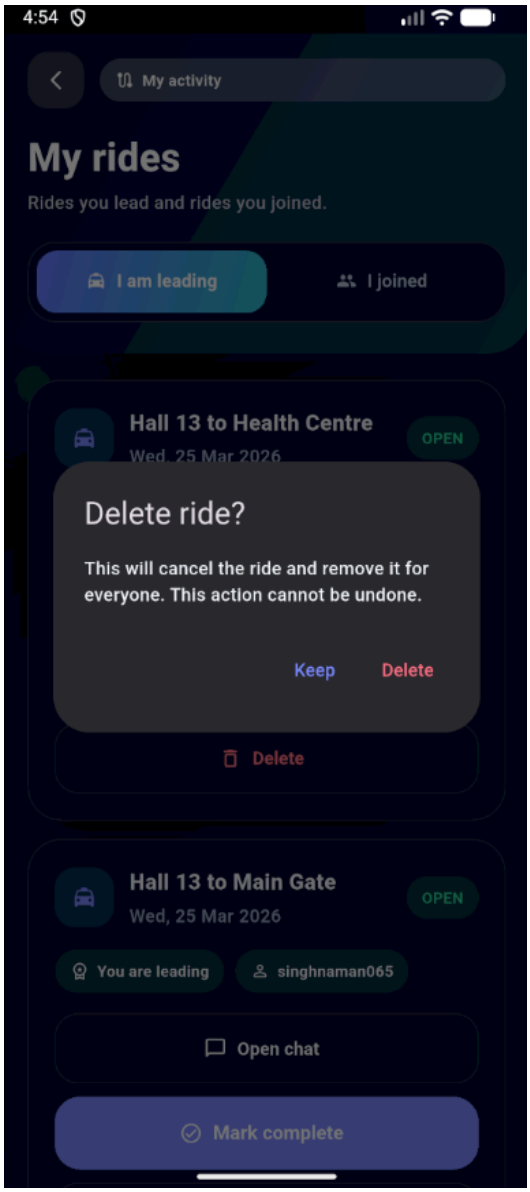
2.3.2.b Delete Ride — Frontend Confirmation

Test Owner: Naman Mohan Singh

Test Date: [21/03/26]

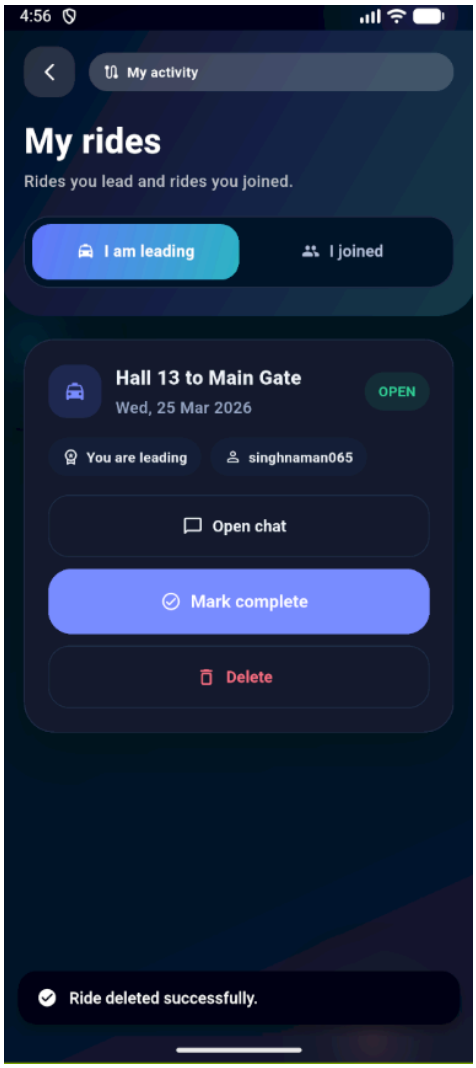
Test Results: Confirmation dialog appeared on every delete tap. Cancelling left the ride unchanged. Confirming removed it and showed "Ride deleted successfully."

Check for delete confirmation dialog:



[My Activity Screen — Delete tapped; AlertDialog shows "Delete ride?" with content warning and "Keep" / "Delete" buttons.]

Check for successful deletion:



[My Activity Screen — After confirming deletion; ride card is removed from list; green snack bar reads "Ride deleted successfully."]

2.3.3.a Query Open Rides with Destination Filter

Firebase SDK Path: FirebaseFirestore.instance.collection('rides').where('status', isEqualTo: 'open').where('destination', isEqualTo: filter)

Test Owner: Naman Mohan Singh

Test Description: Verify that the destination filter correctly excludes non-matching rides and that removing the filter returns all open rides.

Test Results: With filter set to "Hall 1", only rides with destination "Hall 1" were returned. Without the filter, all open rides were returned.

```
// test/rides/filter_rides_test.dart
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('destination filter returns only matching rides', () async {
    final db = FakeFirebaseFirestore();
    await db.collection('rides').add(
        {'status': 'open', 'destination': 'Hall 1'});
    await db.collection('rides').add(
        {'status': 'open', 'destination': 'Main Gate'});

    final filtered = await db.collection('rides')
        .where('status', isEqualTo: 'open')
        .where('destination', isEqualTo: 'Hall 1')
        .get();

    expect(filtered.docs.length, equals(1));
    expect(filtered.docs.first.get('destination'), equals('Hall 1'));
  });
}
```

2.3.4.a Leader Name Fallback Logic

Test Owner: Naman Mohan Singh

Test Description: Verify that the leaderName stored in the ride document correctly falls back to the email prefix when the user's name field in Firestore is empty or not set, matching the logic in CreateRideScreen._submitRide().

Test Results: When name was a non-empty string, it was used as leaderName. When name was empty or null, the fallback to email.split('@').first was applied correctly.

```
// test/rides/leader_name_test.dart
import 'package:flutter_test/flutter_test.dart';

String resolveLeaderName(Map<String, dynamic> data, String? email) {
  final name = (data['name'] as String?)?.trim();
  if (name != null && name.isNotEmpty) return name;
  return email?.split('@').first ?? 'Student';
}

void main() {
  group('LeaderName', () {

    test('uses name field when non-empty', () {
      final name = resolveLeaderName({'name': 'Alice'}, 'alice@iitk.ac.in');
      expect(name, equals('Alice'));
    });

    test('falls back to email prefix when name is empty', () {
      final name = resolveLeaderName({'name': ''}, 'alice@iitk.ac.in');
      expect(name, equals('alice'));
    });

    test('falls back to email prefix when name is null', () {
      final name = resolveLeaderName({}, 'bob@iitk.ac.in');
      expect(name, equals('bob'));
    });

    test('falls back to Student when email is also null', () {
      final name = resolveLeaderName({}, null);
      expect(name, equals('Student'));
    });
  });
}
```

2.4 Chat / Messaging

2.4.1.a Send Message and Register Participant

Firebase SDK Path: rides/{rideId}/messages.add({...}) + rides/{rideId}.update({participants: FieldValue.arrayUnion([uid])})

Test Owner: Naman Mohan Singh

Test Description: Verify that _sendMessage() writes a message document to the messages sub-collection with correct fields and adds the sender's UID to the ride's participants array via arrayUnion.

Test Results: Message written with text, createdAt, senderId, senderEmail. Participants array updated to include sender's UID. Sending again did not duplicate the UID.

```
// test/chat/send_message_test.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('message written and participant registered', () async {
    final db = FakeFirebaseFirestore();
    await db.collection('rides').doc('ride01')
        .set({'participants': []});

    // Simulate _sendMessage()
    await db.collection('rides').doc('ride01')
        .collection('messages').add({
          'text': 'Hello!',
          'senderId': 'uid001',
          'senderEmail': 'alice@iitk.ac.in',
          'createdAt': Timestamp.now(),
        });
    await db.collection('rides').doc('ride01').update({
      'participants': FieldValue.arrayUnion(['uid001']),
    });

    final msgs = await db.collection('rides')
        .doc('ride01').collection('messages').get();
    expect(msgs.docs.length, equals(1));
    expect(msgs.docs.first.get('text'), equals('Hello!'));

    final ride = await db.collection('rides').doc('ride01').get();
    expect(ride.get('participants'), contains('uid001'));
  });
}
```

2.4.1.b Chat — Empty Message Guard and UI

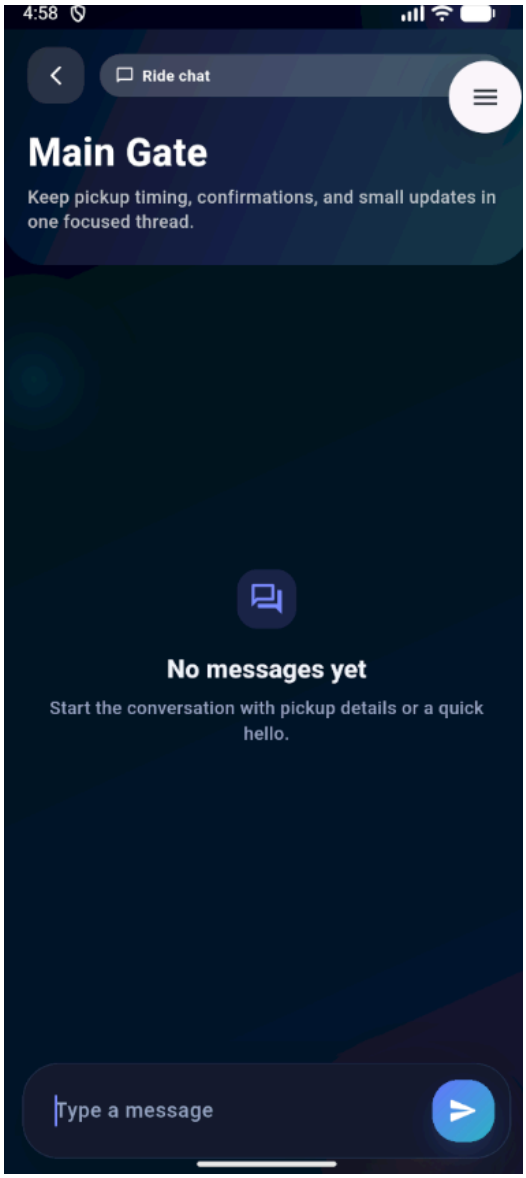
Test Owner: Naman Mohan Singh

Test Date: [22/03/26]

Test Description: Verify that the send action returns early without a Firestore write when the message field is empty or whitespace-only. Verify that sent messages appear right-aligned and received messages appear left-aligned.

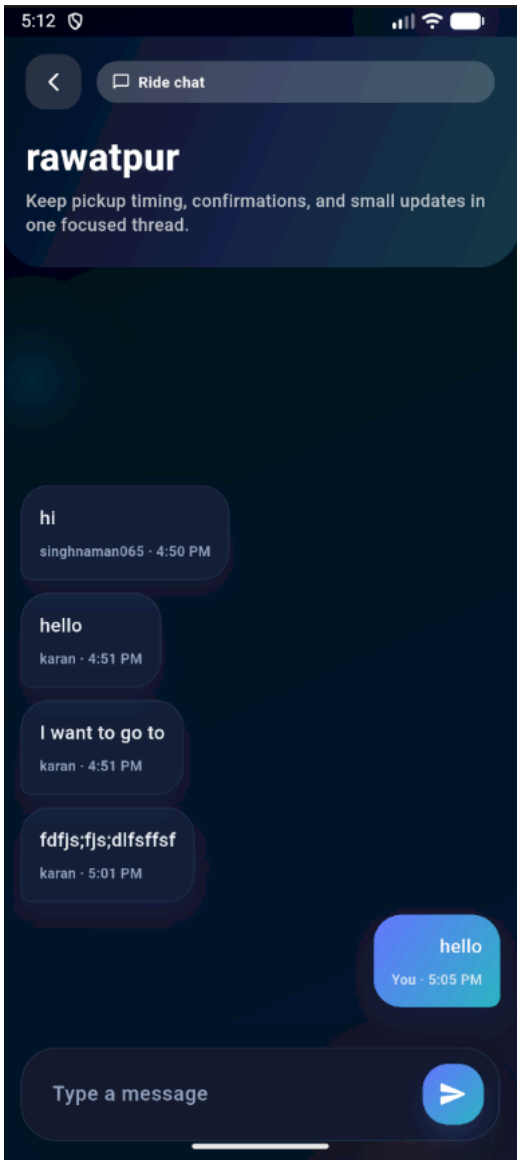
Test Results: Empty/whitespace input caused early return with no Firestore write. Sent messages appeared right-aligned. Received messages appeared left-aligned with the sender's username label.

Check for empty message guard:



[Chat Screen — Send tapped with empty input; no message bubble appears in the list; Firestore messages sub-collection remains unchanged.]

Check for message alignment:



[Chat Screen — "Picking up at 6pm" sent; message bubble appears right-aligned labelled "You". A received message from another user appears left-aligned with their username label.]

2.3.4.a Leader Name Fallback Logic

Test Owner: Naman Mohan Singh

Test Description: Verify that the leaderName stored in the ride document correctly falls back to the email prefix when the user's name field in Firestore is empty or not set, matching the logic in CreateRideScreen._submitRide().

Test Results: When name was a non-empty string, it was used as leaderName. When name was empty or null, the fallback to email.split('@').first was applied correctly.

```
// test/rides/leader_name_test.dart
import 'package:flutter_test/flutter_test.dart';

String resolveLeaderName(Map<String, dynamic> data, String? email) {
  final name = (data['name'] as String?)?.trim();
  if (name != null && name.isNotEmpty) return name;
  return email?.split('@').first ?? 'Student';
}

void main() {
  group('LeaderName', () {

    test('uses name field when non-empty', () {
      final name = resolveLeaderName({'name': 'Alice'}, 'alice@iitk.ac.in');
      expect(name, equals('Alice'));
    });

    test('falls back to email prefix when name is empty', () {
      final name = resolveLeaderName({'name': ''}, 'alice@iitk.ac.in');
      expect(name, equals('alice'));
    });

    test('falls back to email prefix when name is null', () {
      final name = resolveLeaderName({}, 'bob@iitk.ac.in');
      expect(name, equals('bob'));
    });

    test('falls back to Student when email is also null', () {
      final name = resolveLeaderName({}, null);
      expect(name, equals('Student'));
    });
  });
}
```

3- INTEGRATION TESTING

Integration testing validates whether the Flutter frontend and Firebase backend interact correctly and produce the desired end-to-end behaviour. Both components were individually verified through unit testing before integration testing commenced.

Testing was performed by supplying input through the application UI and observing the resulting changes in Firestore via the Firebase Emulator Suite console. Correct final behaviour confirms that the frontend-Firebase interaction was correct.

3.1 Authentication

Module Details:

End-to-end testing of registration with email verification enforcement, sign-in with valid and invalid credentials, Google OAuth, password reset, and reactive session routing between AuthScreen and HomeScreen.

Test Date: [24/03/26]

Test Results:

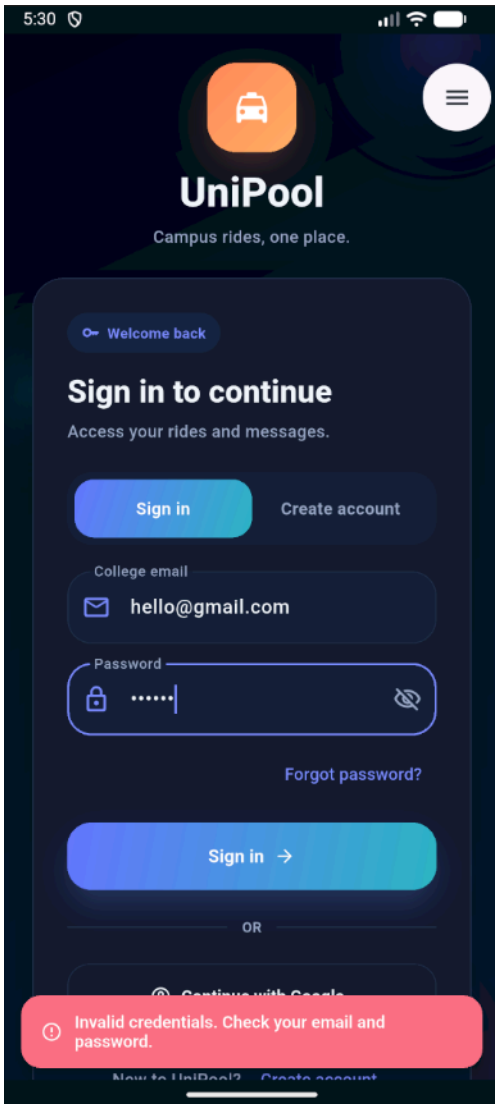
Email verification was correctly enforced at sign-in. The authStateChanges() stream correctly drove navigation between screens. Firestore user documents were created on first registration and first Google sign-in.

APIs tested in this module:

- signInWithEmailAndPassword
- createUserWithEmailAndPassword + sendEmailVerification
- sendPasswordResetEmail
- signInWithCredential (Google OAuth)
- signOut

TEST #1

Details: Invalid email address entered on sign-in screen.

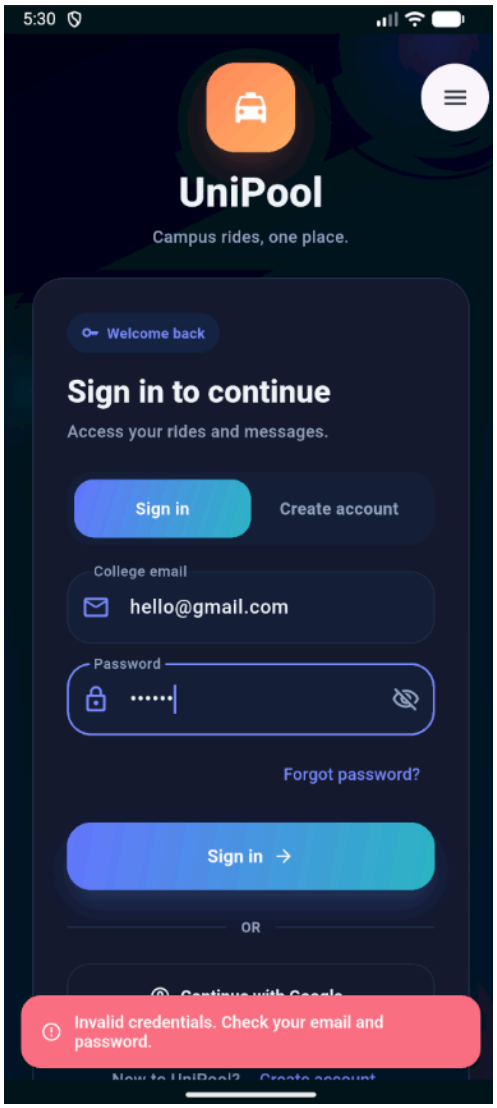


[I Authentication Screen — Non-existent email entered; snack bar reads "Invalid Credentials"]

Result: Firebase Auth exception code user-not-found was caught. Descriptive error shown. Sign-in denied.

TEST #2

Details: Incorrect password entered for a registered account.

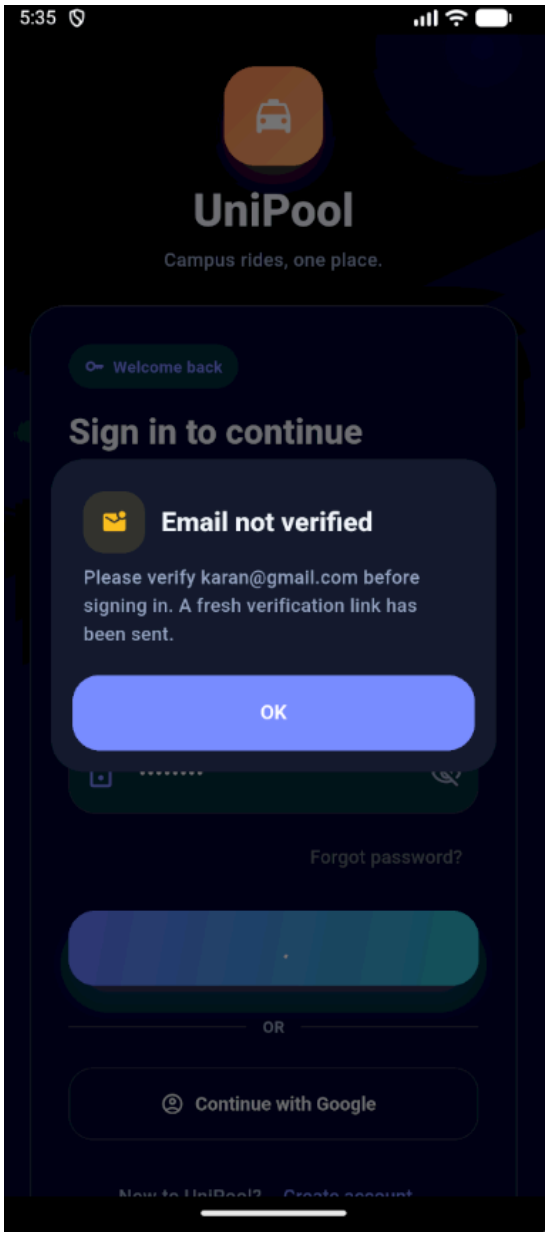


[Authentication Screen — Correct email with wrong password; snack bar reads "Invalid credentials. Check your email and password."]

Result: Firebase Auth exception code invalid-credential was caught and displayed. Sign-in denied.

TEST #3

Details: Sign-in attempted with an account whose email has not been verified.



[Authentication Screen — Valid credentials entered; email unverified; snack bar reads "Verify your email before signing in. A fresh link has been sent."]

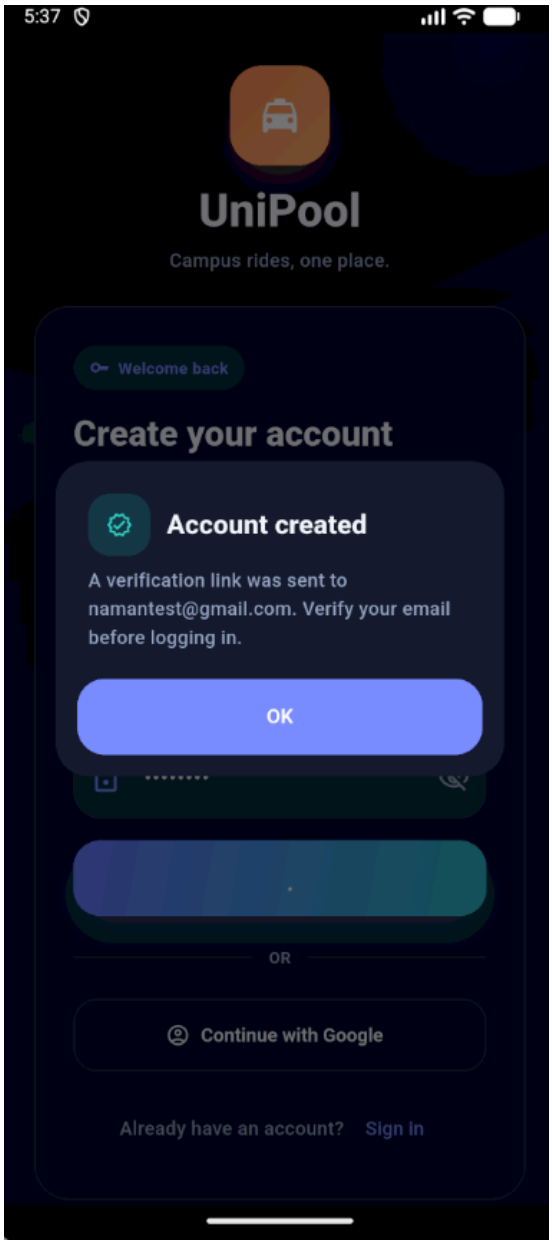
Result: The emailVerified check after signInWithEmailAndPassword() identified the unverified state, called signOut(), and re-dispatched a verification email. Access to HomeScreen denied.

TEST #4

Details: Successful registration with email/password.

Flow:

- User enters valid email and a password ≥6 characters on the Create Account tab.
- createUserWithEmailAndPassword() called; verification email dispatched.
- User signed out; AuthScreen switches to Sign In tab.
- Firestore users/{uid} document created with name, email, ridesCompleted: 0, createdAt.
- User verifies email, signs in; HomeScreen displayed.

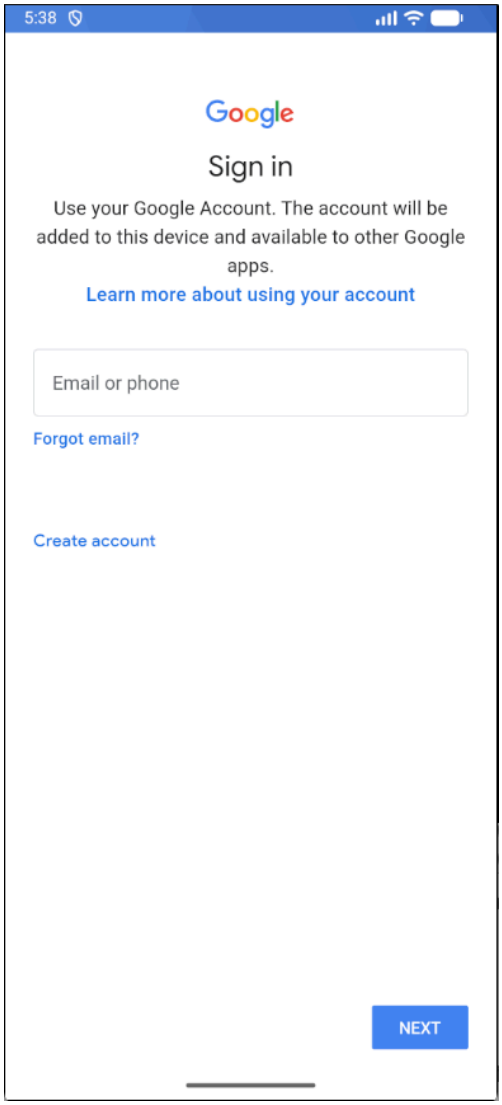


[Authentication Screen — After sign-up; snack bar reads "Account created. Verify your email before logging in." Screen switches to Sign In tab.]

Result: Registration succeeded end-to-end. Firestore document created. Post-verification sign-in navigated to HomeScreen.

TEST #5

Details: Google Sign-In flow for a first-time user.

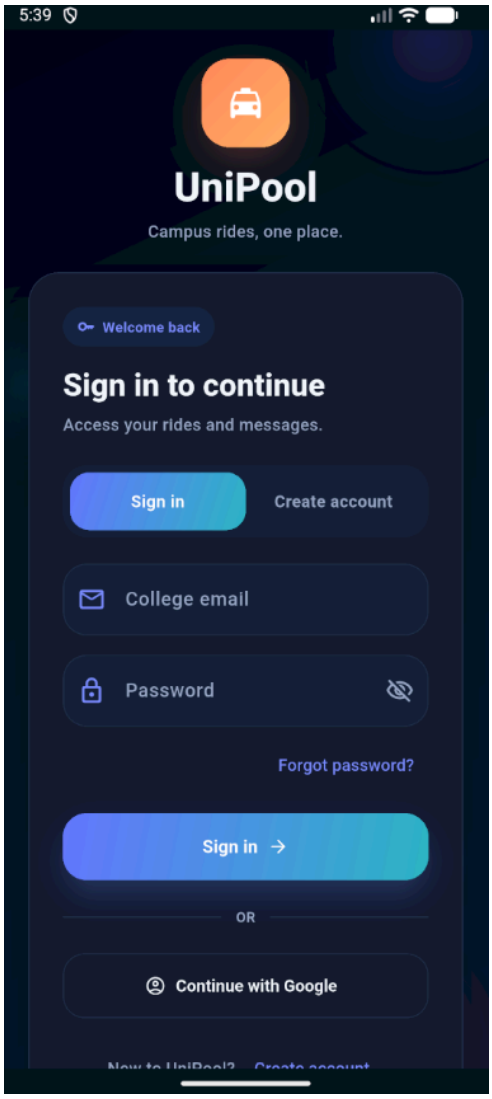


[Authentication Screen — "Continue with Google" tapped; native Google account picker appears; after selection user is redirected to HomeScreen and a new Firestore document is created under users/{uid}.]

Result: Firebase credentials were constructed from Google tokens. signInWithCredential() succeeded. Firestore user document was created for the new UID.

TEST #6

Details: Successful sign-out from HomeScreen.



[Home Screen — Sign-out icon tapped; user immediately redirected to AuthScreen; session token cleared.]

Result: FirebaseAuth.instance.signOut() cleared the session. authStateChanges() stream emitted null, triggering navigation to AuthScreen.

3.2 Create Ride Post

Module Details:

End-to-end testing of ride creation, including field validation, Firestore document creation, and immediate visibility on the Find Ride screen.

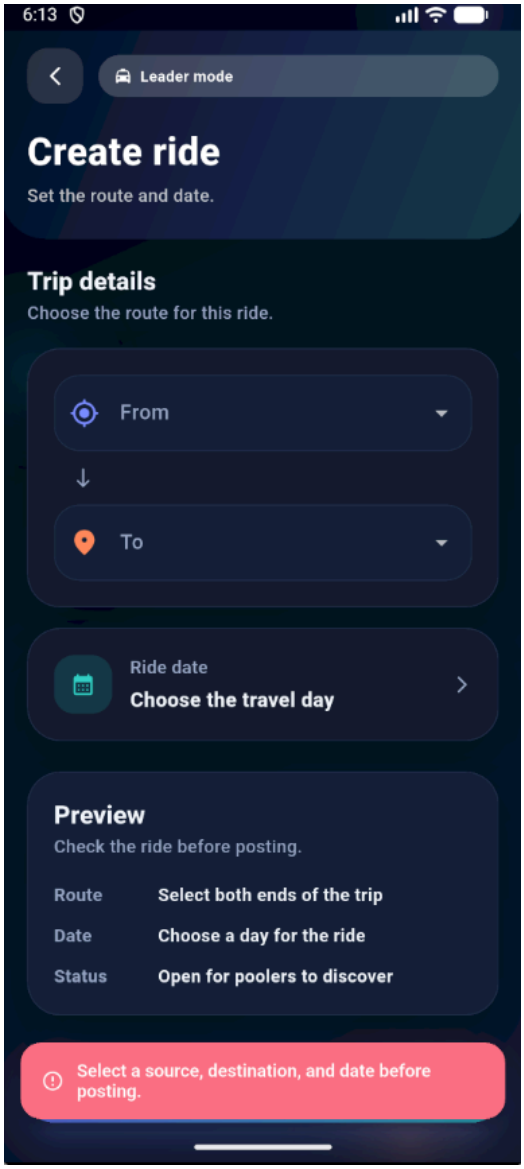
Test Date: [26/03/26]

Test Results:

Ride documents appeared immediately in the FindRideScreen StreamBuilder. All field validations functioned correctly. The Firestore document schema matched the expected structure.

TEST #1

Details: Post tapped with no source, destination, or date selected.

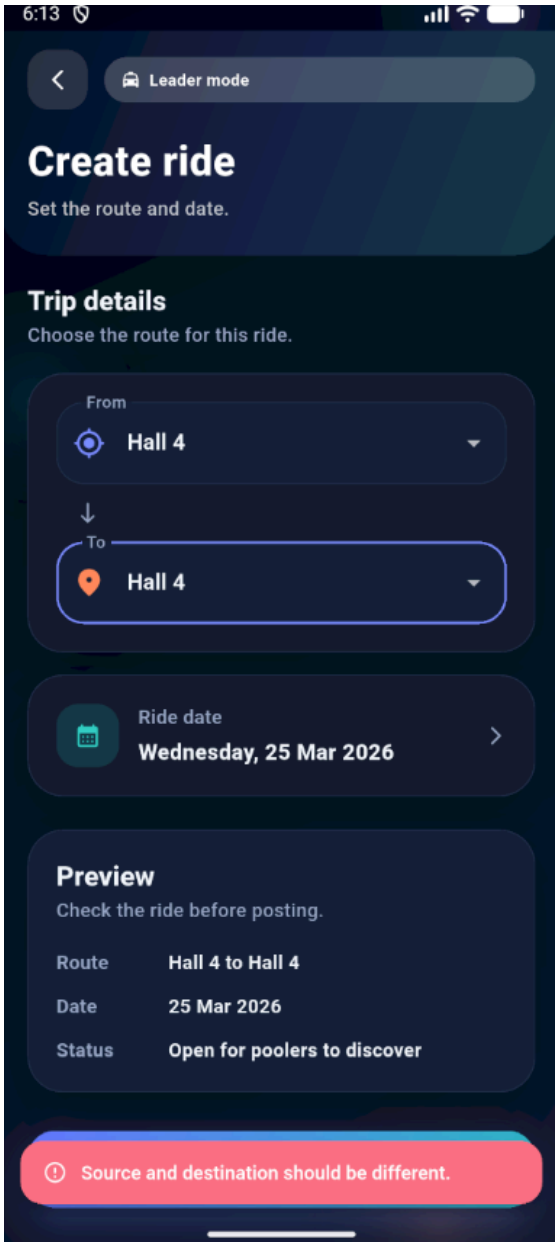


[Create Ride Screen — Post button tapped with all fields empty; snack bar reads "Select a source, destination, and date before posting."]

Result: Client-side validation prevented the Firestore write.

TEST #2

Details: Same location selected for both source and destination.



[Create Ride Screen — "Hall 1" selected for both fields; snack bar reads "Source and destination should be different."]

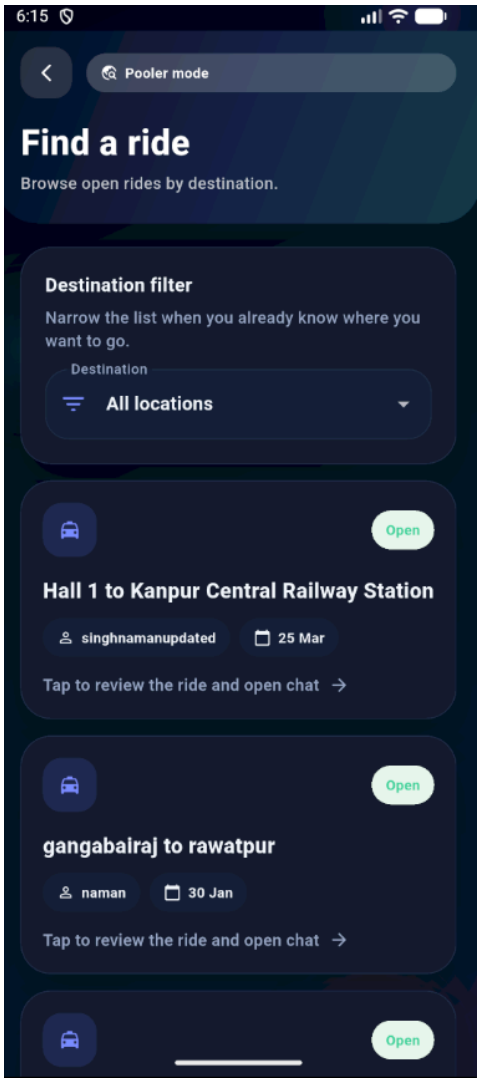
Result: Client-side guard blocked the write.

TEST #3

Details: Valid ride posted with all fields populated.

Flow:

- Source "Hall 1" selected from dropdown.
- Destination "Kanpur Central Railway Station" selected.
- Future date selected via date picker.
- "Post ride request" tapped.
- Firestore ride document created with status "open" and participants [].
- User returned to HomeScreen with snack bar "Ride posted successfully."
- New ride card appears immediately on FindRideScreen for other users.



[Find Ride Screen — Immediately after post; new ride card "Hall 1 to Kanpur Central Railway Station" appears at top of live ride list with posting user's name and formatted date.]

Result: Ride document created in Firestore with all expected fields. Ride appeared immediately in the FindRideScreen StreamBuilder for all connected users.

3.3 Find and Filter Rides

Module Details:

End-to-end testing of ride discovery, real-time updates, destination filtering, ride detail bottom sheet, and Join and Chat navigation.

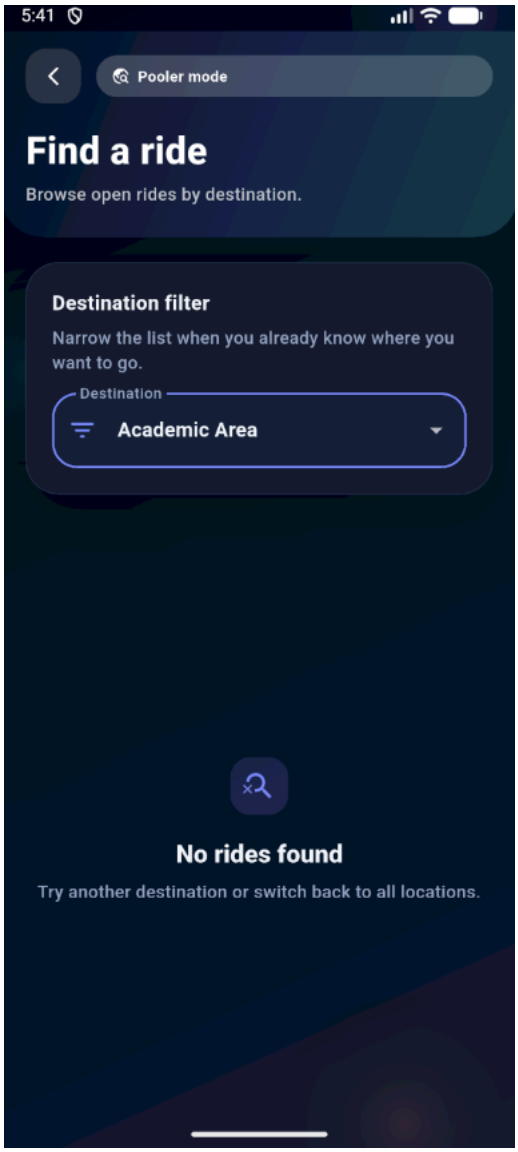
Test Date: [26/03/26]

Test Results:

StreamBuilder reflected live additions and removals. Destination filtering returned only matching rides. The ride detail bottom sheet displayed all fields correctly. Join and Chat navigation functioned end-to-end.

TEST #1

Details: No rides exist in the database with chosen destination.

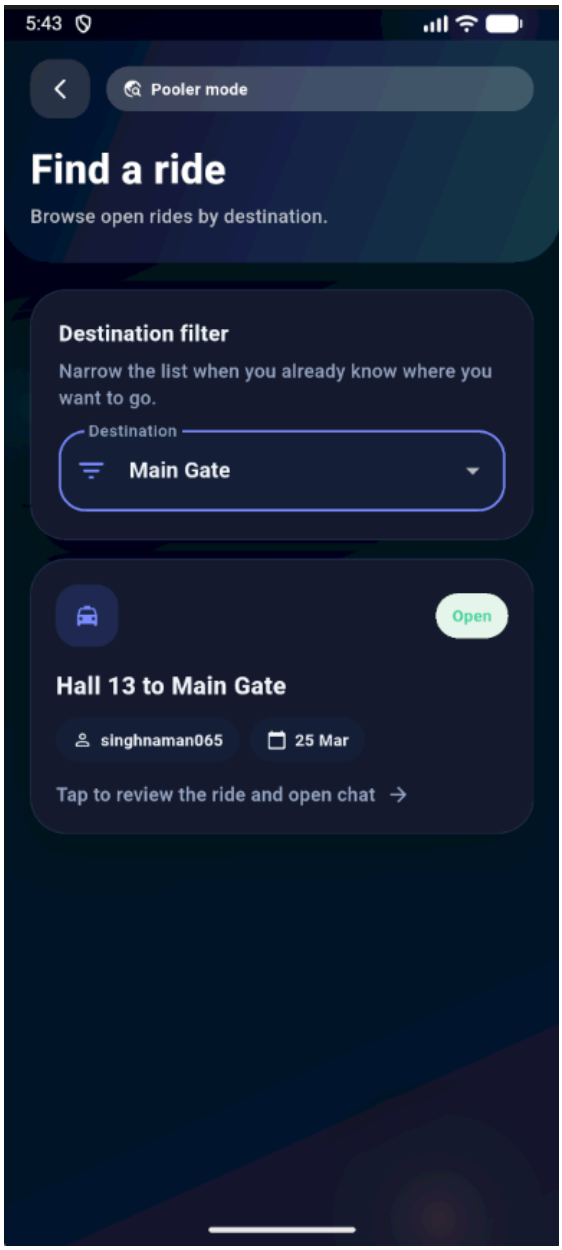


[Find Ride Screen — Empty state shown with icon and text "No rides found. There are no open rides right now."]

Result: StreamBuilder handled the empty snapshot and displayed AppEmptyState correctly.

TEST #2

Details: Destination filter applied; only matching rides shown.

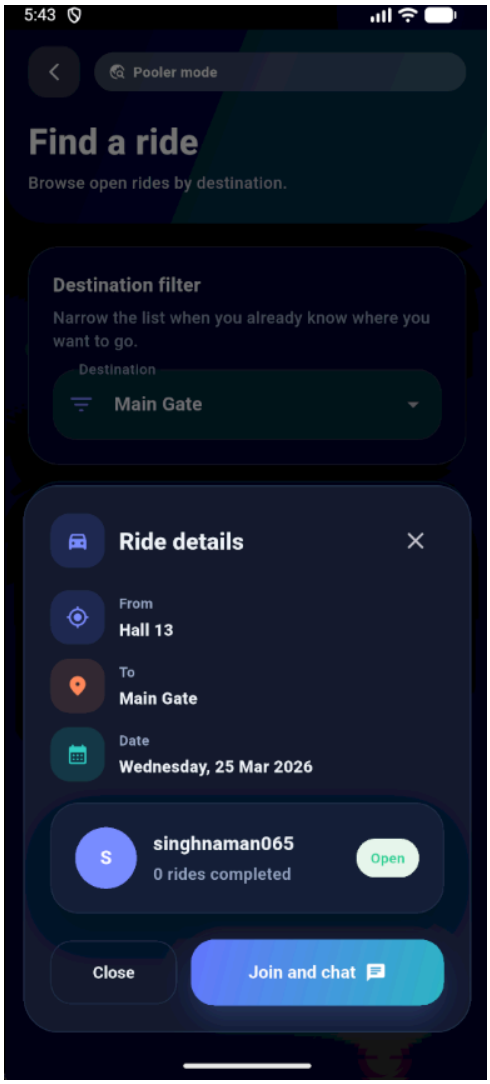


[Find Ride Screen — Destination dropdown set to "Main Gate"; only rides with destination "Main Gate" visible; other rides hidden.]

Result: Compound Firestore query filtered correctly.

TEST #3

Details: Ride card tapped; detail bottom sheet with leader trust info appears.

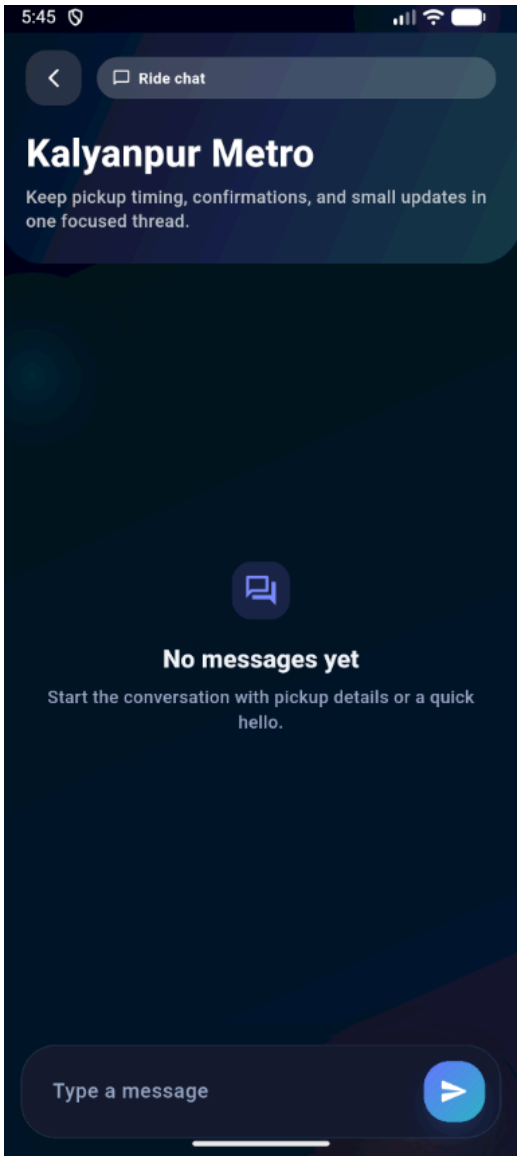


[Find Ride Screen — Ride card tapped; modal bottom sheet slides up showing source, destination, date, leader name, rides-completed count, status pill, and "Join and Chat" button.]

Result: Bottom sheet fetched leader's Firestore document and displayed ridesCompleted count correctly.

TEST #4

Details: "Join and Chat" tapped from detail sheet; navigates to ChatScreen.



[Chat Screen — Navigated from "Join and Chat"; empty chat with "No messages yet" empty state; rideDestination shown in header; message input ready.]

Result: Navigation passed correct rideId and rideDestination. StreamBuilder subscribed to correct sub-collection.

3.4 Edit Profile

Module Details:

End-to-end testing of profile data loading, display name update, photo upload to Firebase Storage, and Firestore reflection of changes.

Test Owner: Naman Mohan Singh

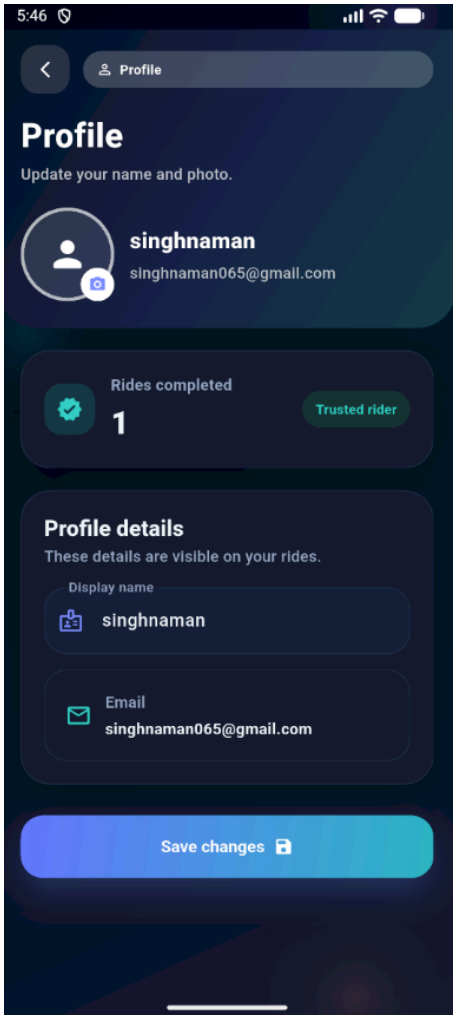
Test Date: [27/03/26]

Test Results:

Existing profile data loaded correctly from Firestore. Display name changes persisted via merge write. Photo uploads to Firebase Storage succeeded and CDN URL was stored in Firestore.

TEST #1

Details: ProfileScreen opened; existing data loaded from Firestore.

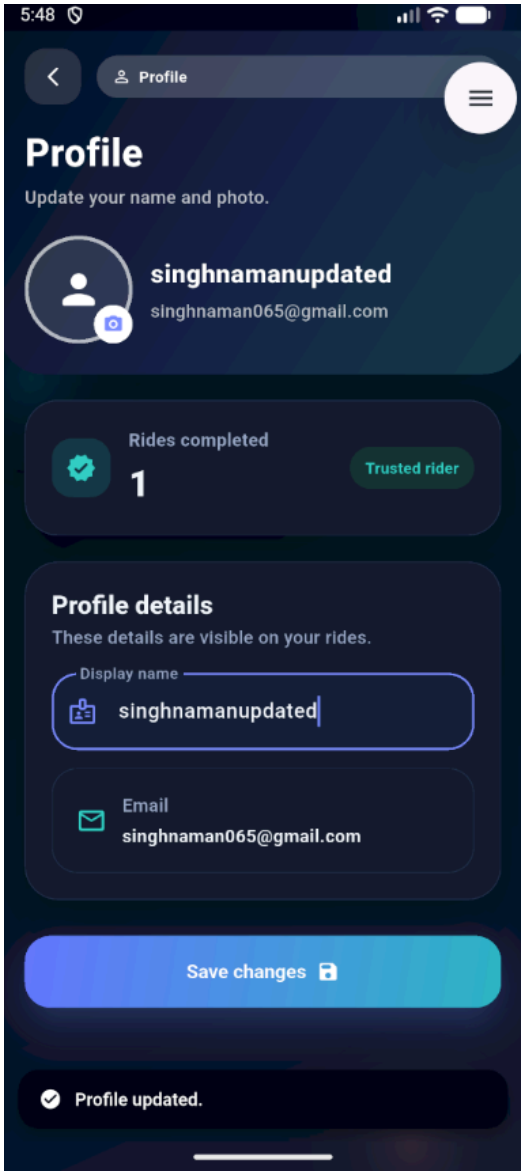


[Profile Screen — Screen opens with previously saved display name in field and previously uploaded photo in avatar; ridesCompleted count shown below.]

Result: _loadUserData() correctly populated all local state variables from Firestore.

TEST #2

Details: Display name edited and saved.



[Profile Screen — Name field updated; "Save Changes" tapped; snack bar reads "Profile updated."]

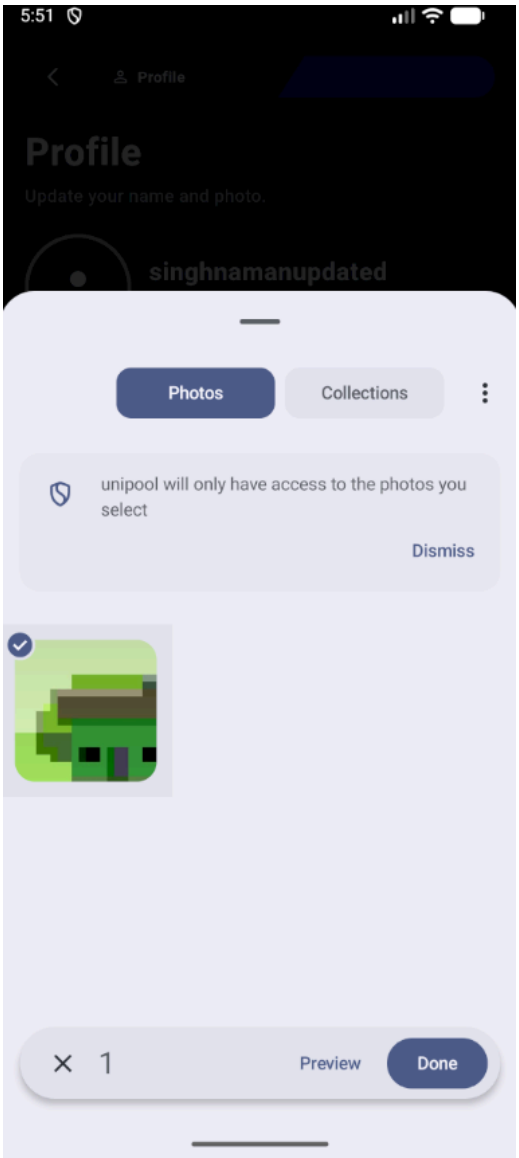
Result: Merge write updated only the name field. Updated name immediately reflected in the header.

TEST #3

Details: Profile photo selected from gallery and uploaded to Firebase Storage.

Flow:

- User taps avatar circle.
- Device gallery opens via image_picker.
- Photo selected; uploaded to Firebase Storage at user_images/{uid}.jpg.
- CDN URL returned and written to Firestore users/{uid} as photoUrl.



[Profile Screen — New photo selected; progress indicator overlaid on avatar during upload; new photo could be chosen]

Result: Upload completed. Firestore photoUrl updated.

3.5 Chat System

Module Details:

End-to-end testing of per-ride group chat, real-time message delivery, participant registration via arrayUnion, and message alignment logic.

Test Owner: Naman Mohan Singh

Test Date: [27/03/26]

Test Results:

Messages from one user appeared immediately on another user's screen. The participants array updated correctly on first message send. Sent messages appeared right-aligned; received left-aligned.

TEST #1

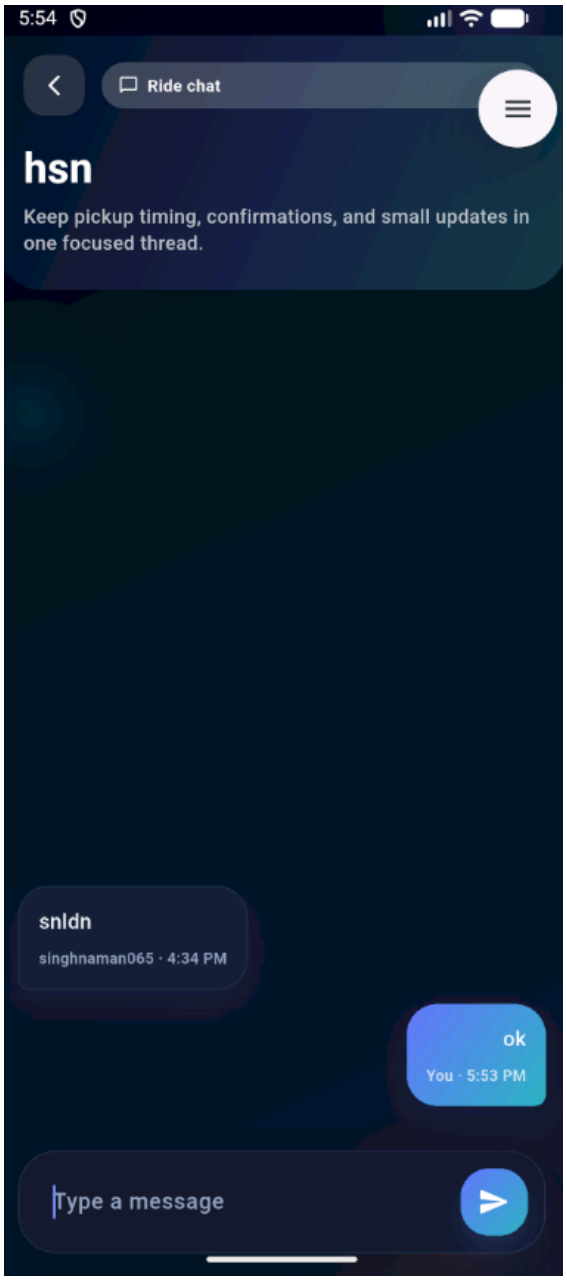
Details: Successful text message send.

[Chat Screen — "Picking up at 6pm" sent; message bubble appears right-aligned with label "You" and formatted timestamp.]

Result: Message document written to sub-collection with correct fields. Message appeared immediately in the list.

TEST #2

Details: Message from another user appears in real time.



[Chat Screen — Second user sends a message from another device; message bubble appears left-aligned immediately with their username and timestamp.]

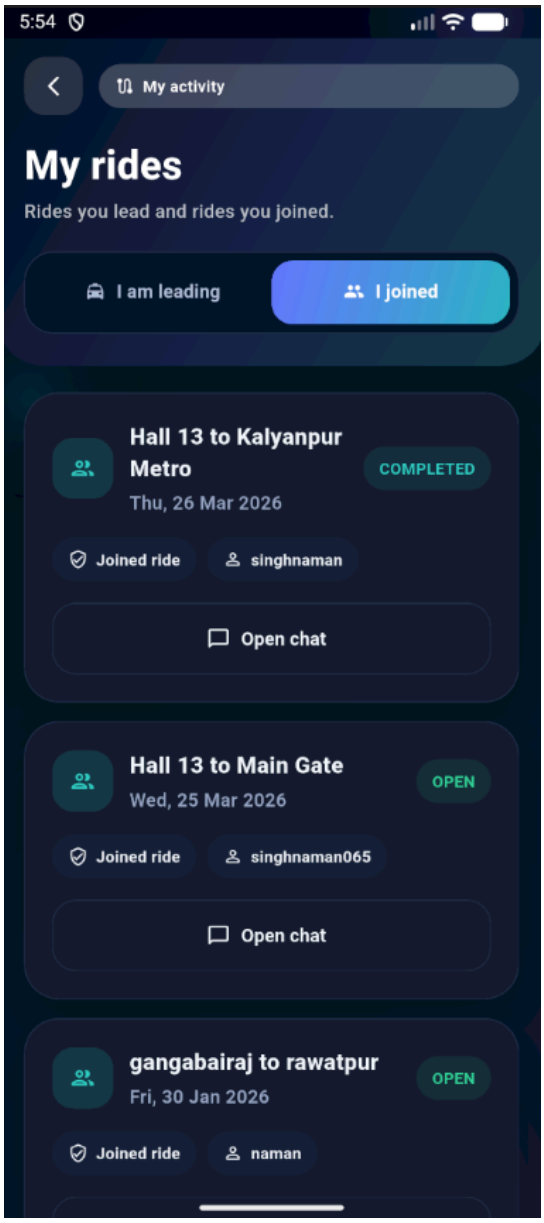
Result: StreamBuilder delivered the new message document to all subscribed listeners without screen refresh.

TEST #3

Details: Sending a message registers the user as a participant.

Flow:

- User joins chat via "Join and Chat" from FindRideScreen.
- User sends a message.
- Firestore ride document's participants array updated via arrayUnion([uid]).
- Ride appears in user's "I Joined" tab on MyRidesScreen.

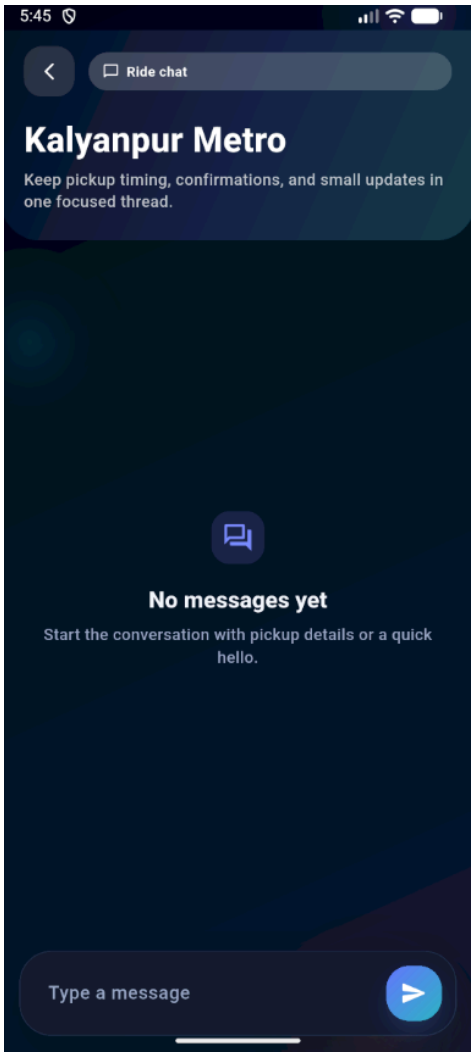


[My Activity Screen — "I Joined" tab; ride joined via chat now visible in list after first message sent.]

Result: arrayUnion correctly added UID to participants. Ride appeared in correct tab immediately.

TEST #4

Details: Empty message blocked from sending.

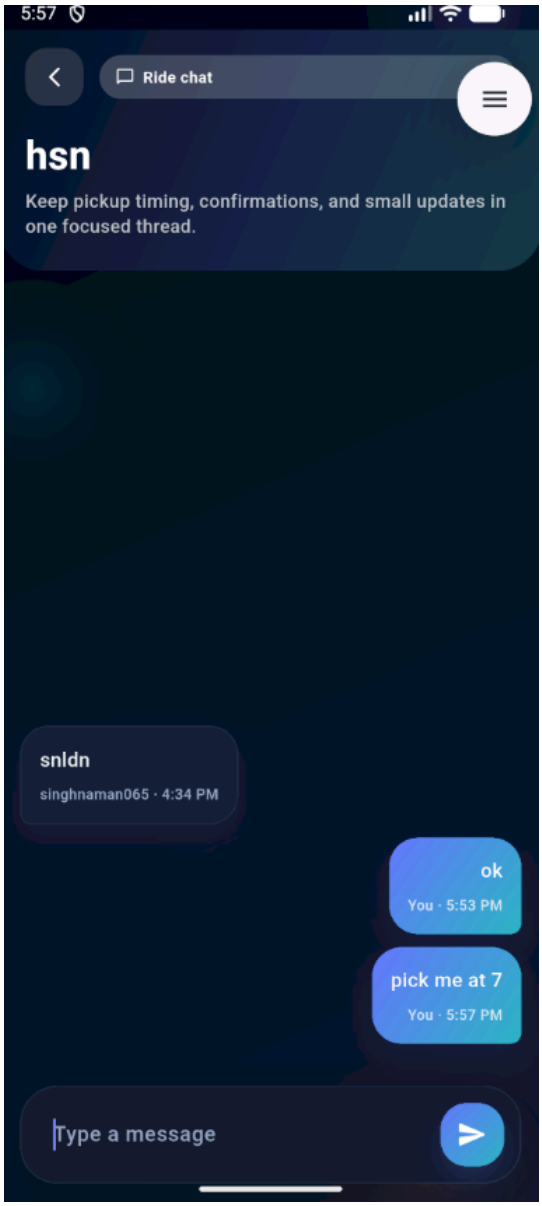


[Chat Screen — Send tapped with empty input; no new message bubble appears; no Firestore write triggered.]

Result: Early return in _sendMessage() when message.trim().isEmpty blocked both the Firestore write and the arrayUnion call.

TEST #5

Details: Messages are displayed in ascending chronological order (oldest at top, newest at bottom).

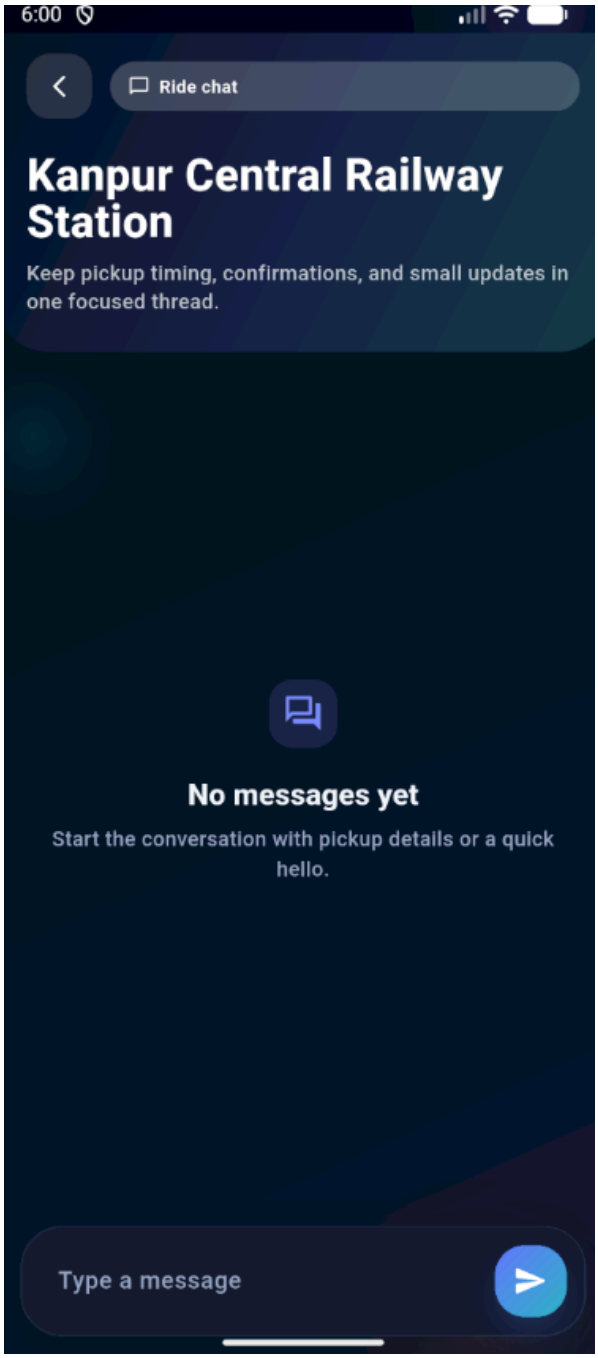


[Chat Screen — Three messages sent at different times; messages appear in the list ordered oldest to newest from top to bottom; the latest message is visible at the bottom without scrolling.]

Result: The StreamBuilder query uses orderBy('createdAt', descending: false) on the messages sub-collection. Messages are rendered in correct chronological order.

TEST #6

Details: Chat screen header displays the correct rideDestination passed as a constructor parameter.



[Chat Screen — Chat opened for a ride to "Kanpur Central Railway Station"; page header shows "Kanpur Central Railway Station" as the title and "Ride chat" pill badge in the sub-header.]

Result: The rideDestination parameter was correctly passed from FindRideScreen and displayed in AppPageHeader. The title matched the ride's destination field in Firestore.

3.6 My Activity

Module Details:

This module covers end-to-end testing of the MyRidesScreen activity view, including tab switching between rides led and rides joined, the mark-complete flow with Firestore state update and counter increment, the delete-with-confirmation flow, status pill accuracy, and the Open Chat navigation from a ride card.

Test Owner: Naman Mohan Singh & Karan Kumar

Test Date: [30/03/26]

Test Results:

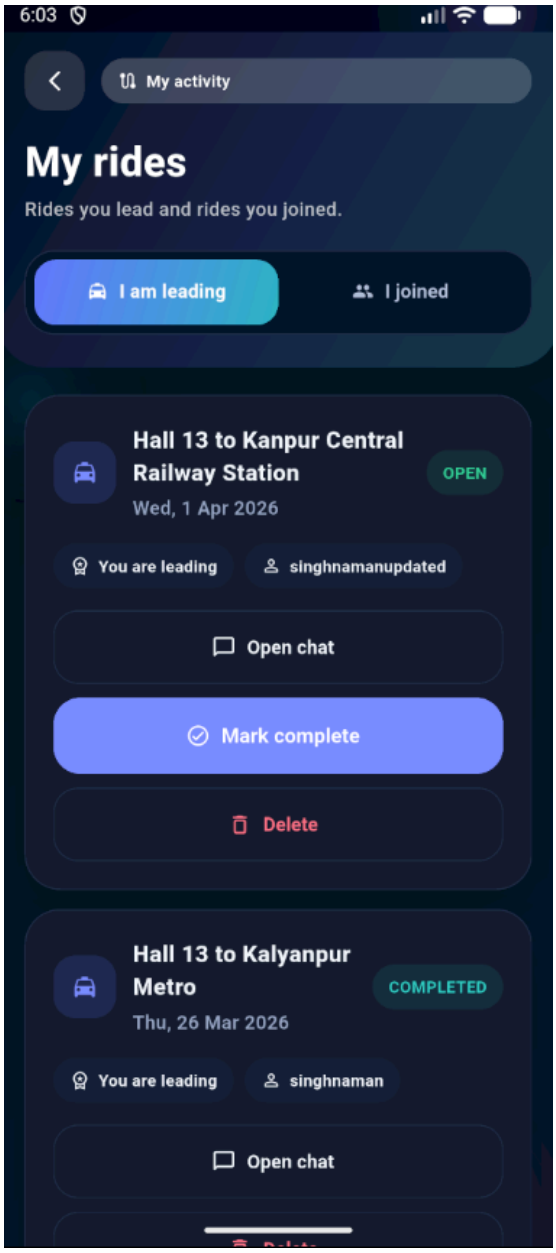
The "I Am Leading" tab correctly displayed only rides where the current user was the leader. The "I Joined" tab correctly reflected rides where the user had sent a chat message. Mark-complete updated the Firestore ride status and the leader's ridesCompleted counter atomically. The delete flow removed the document and updated all live views. Status pills displayed the correct colour for each state.

Firestore operations tested in this module:

- rides.where('leaderId', isEqualTo: uid) — for I Am Leading tab
- rides.where('participants', arrayContains: uid) — for I Joined tab
- rides/{rideId}.update({'status': 'completed'}) — for mark complete
- users/{uid}.update({'ridesCompleted': FieldValue.increment(1)}) — for trust counter
- rides/{rideId}.delete() — for ride deletion

TEST #1

Details: My Activity screen opened; "I Am Leading" tab shows only rides posted by the current user.

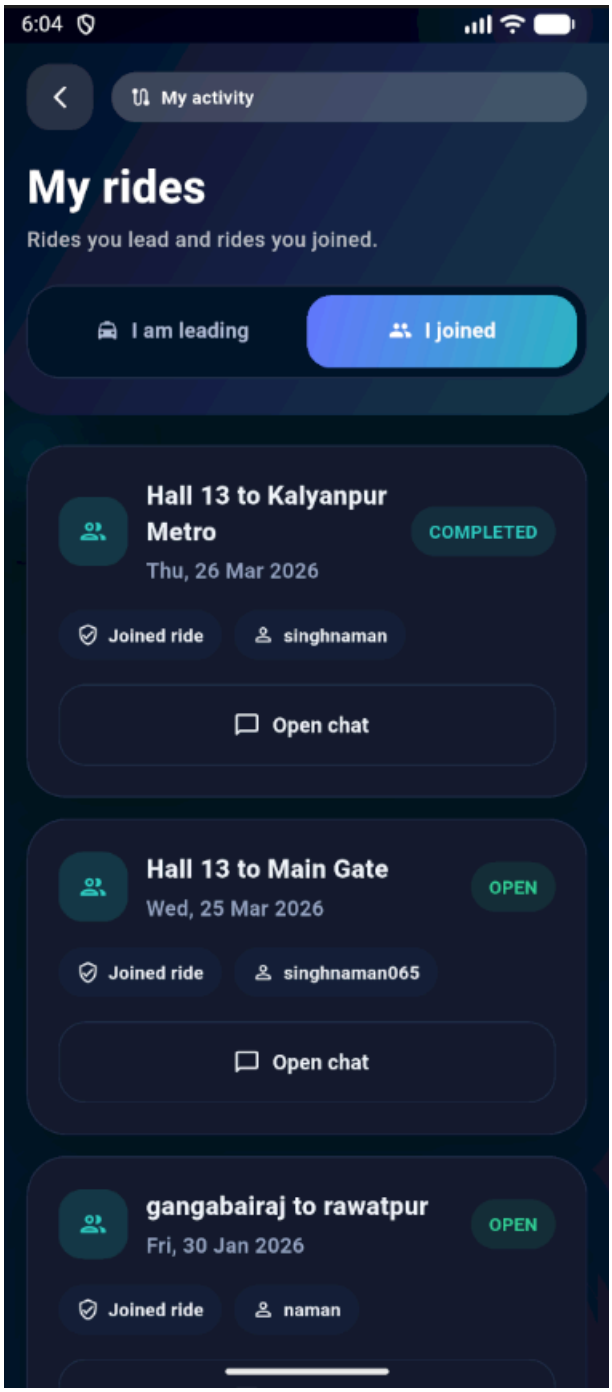


[My Activity Screen — "I Am Leading" tab selected (highlighted with accentGradient indicator); two ride cards visible, both with source/destination matching rides posted by the signed-in user; each card shows an "Open chat", "Mark complete", and "Delete" button.]

Result: The Firestore query .where('leaderId', isEqualTo: user.uid) returned only rides created by the current user. Rides posted by other users did not appear in this tab.

TEST #2

Details: "I Joined" tab shows rides where the user has sent a chat message.

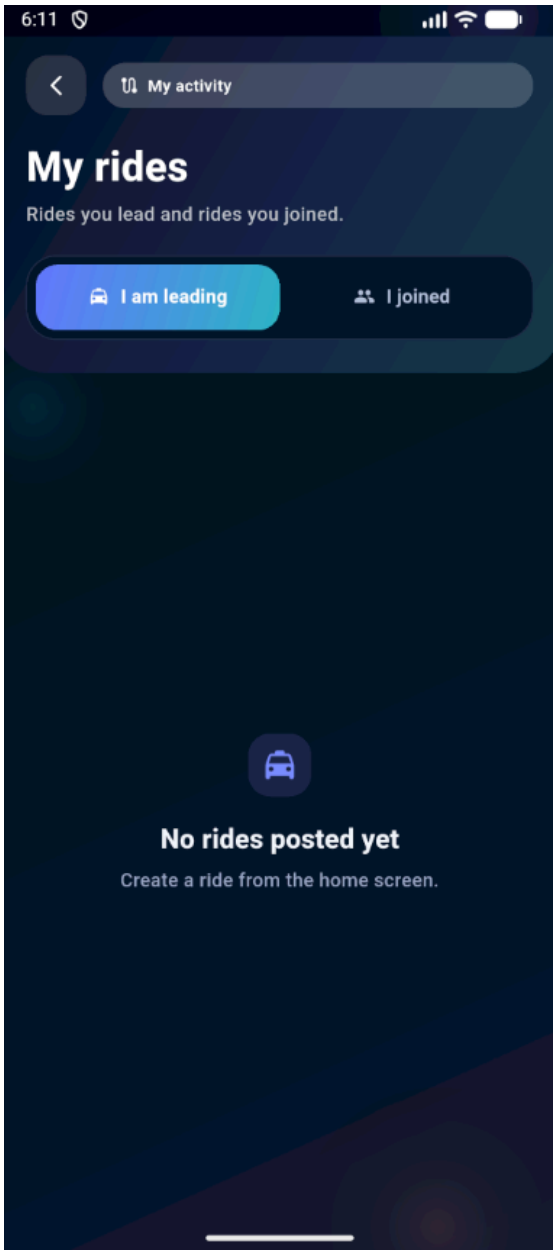


[My Activity Screen — "I Joined" tab selected; one ride card visible for a ride the user joined via chat; card shows "Open chat" button but no "Mark complete" or "Delete" buttons (not the leader).]

Result: The Firestore query .where('participants', arrayContains: user.uid) returned rides where the user's UID was in the participants array. The "Mark complete" and "Delete" actions were correctly absent for non-leader rides.

TEST #3

Details: "I Am Leading" tab shows empty state when no rides have been posted.



[My Activity Screen — "I Am Leading" tab; no ride cards visible; empty state icon and text reads "No rides posted yet. Create a ride from the home screen."]

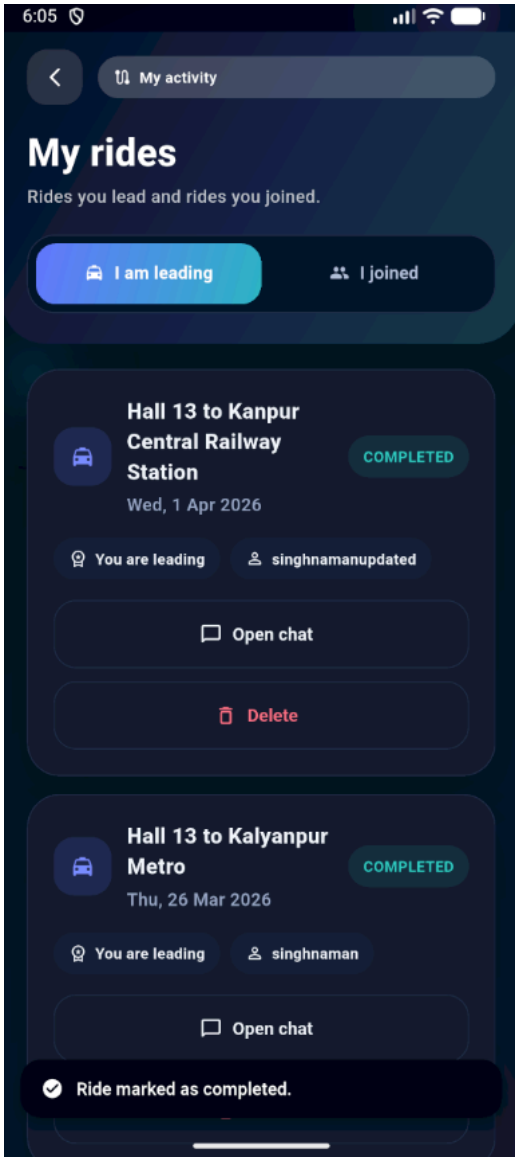
Result: The StreamBuilder correctly handled an empty query snapshot and rendered the AppEmptyState widget with the correct icon and message.

TEST #4

Details: "Mark complete" tapped on an open ride; Firestore status updated and ridesCompleted incremented.

Flow:

- Ride leader navigates to MyRidesScreen, "I Am Leading" tab.
- Open ride card shows "Mark complete" button.
- "Mark complete" tapped.
- Firestore rides/{rideId} status field updated from "open" to "completed".
- Firestore users/{leaderId} ridesCompleted incremented by 1 via FieldValue.increment(1).
- Ride card's status pill changes from green "OPEN" to teal "COMPLETED".
- "Mark complete" button disappears from the card.
- Green snack bar reads "Ride marked as completed."

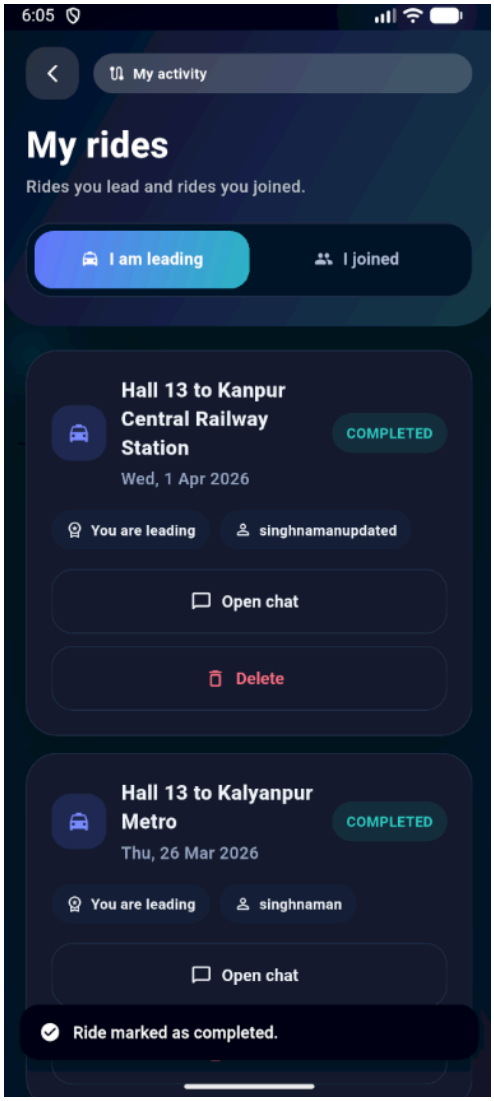


[My Activity Screen — "Mark complete" tapped; ride card status pill changes from green "OPEN" to teal "COMPLETED"; "Mark complete" button disappears; snack bar reads "Ride marked as completed."]

Result: Both Firestore writes succeeded atomically. The UI updated immediately via the StreamBuilder. The ridesCompleted counter in ProfileScreen reflected the incremented value after navigating there.

TEST #5

Details: Attempting to mark a ride that has already been completed — "Mark complete" button is absent.

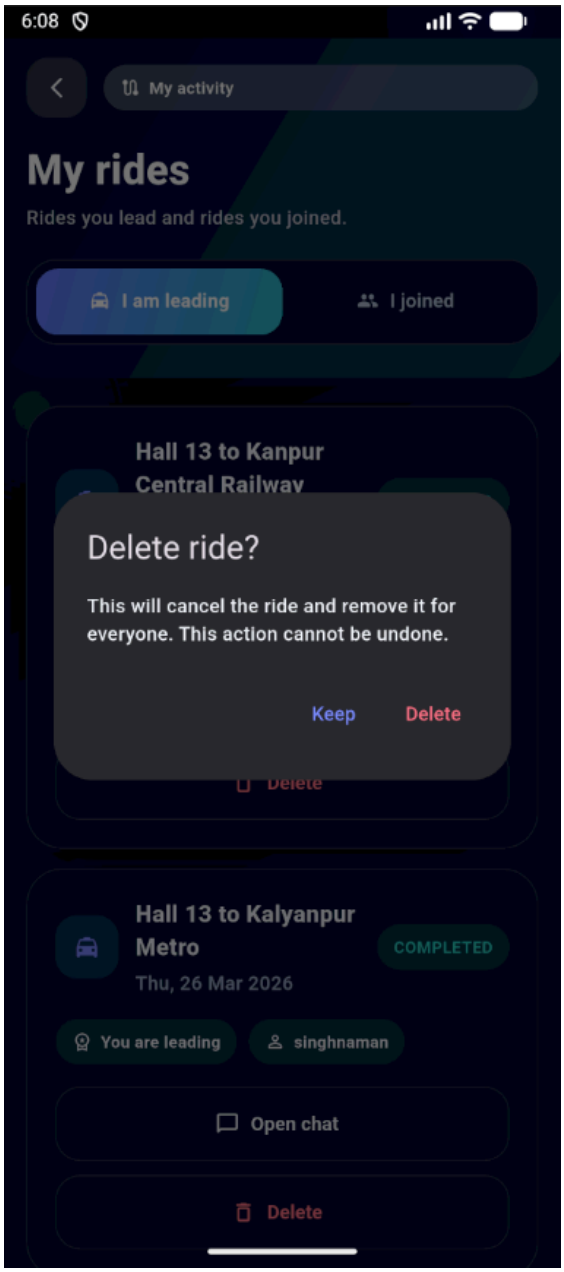


[My Activity Screen — Completed ride card shows "COMPLETED" status pill; no "Mark complete" button is visible; only "Open chat" and "Delete" buttons are shown.]

Result: The "Mark complete" button is conditionally rendered only when isLeader && status == 'open'. For completed rides, the button is absent, preventing a double-increment of the ridesCompleted counter.

TEST #6

Details: "Delete" tapped on an open ride; confirmation dialog appears; cancelling preserves the ride.



[My Activity Screen — "Delete" tapped; AlertDialog with title "Delete ride?" and message body appears; "Keep" and "Delete" buttons present; tapping "Keep" dismisses dialog; ride card remains in the list unchanged.]

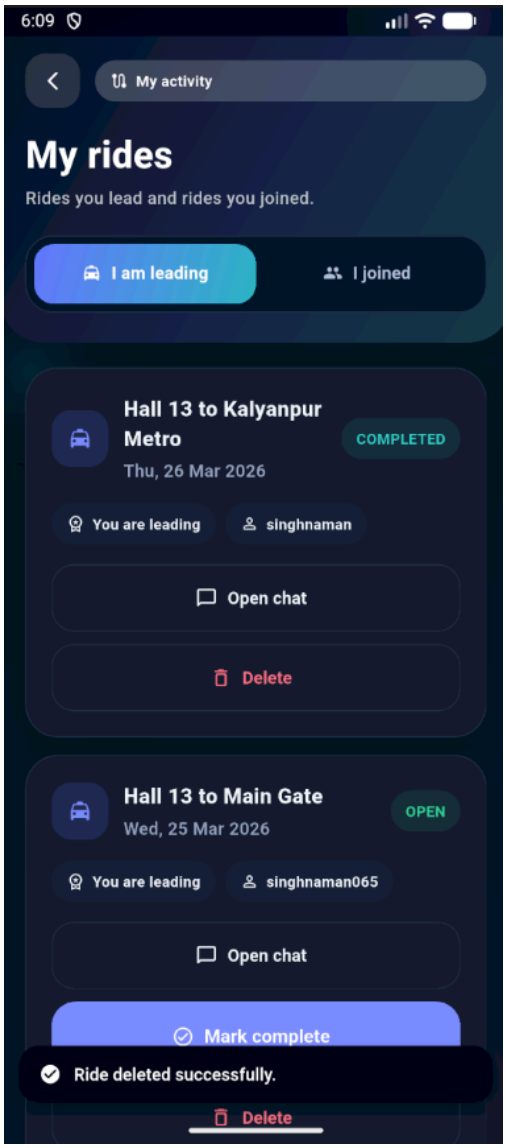
Result: The AlertDialog was presented via showDialog(). Returning false from the dialog (tapping "Keep") caused _deleteRide() to return early without a Firestore write. Ride card remained.

TEST #7

Details: "Delete" confirmed; ride removed from Firestore and all live views.

Flow:

- "Delete" tapped on a ride card in "I Am Leading" tab.
- Confirmation dialog appears.
- "Delete" button in dialog confirmed.
- Firestore rides/{rideId}.delete() called.
- Ride card disappears from the "I Am Leading" tab immediately.
- Ride no longer appears on FindRideScreen for any user.
- Green snack bar reads "Ride deleted successfully."



[My Activity Screen — After deletion confirmed; ride card is gone from the list; snack bar reads "Ride deleted successfully."]

Result: The Firestore delete call removed the document. The StreamBuilder on both MyRidesScreen and FindRideScreen immediately reflected the removal, confirming real-time propagation of the delete to all live queries.

TEST #8

Details: "Open chat" button on a ride card in My Activity navigates to the correct ChatScreen.

[My Activity Screen — "Open chat" tapped on a ride card; navigates to ChatScreen with the correct ride destination displayed in the header and the correct messages sub-collection loaded.]

Result: MaterialPageRoute push correctly passed rideId and rideDestination to ChatScreen. The StreamBuilder subscribed to the correct rides/{rideId}/messages sub-collection and existing messages were displayed.

4 SYSTEM TESTING

System testing validates that the complete, integrated UniPool application satisfies the functional requirements from the Software Requirements Specification.

4.1.1 Account Setup and Authentication

1) Requirement: Only authenticated users with a valid Firebase Auth session can access the application.

Test Owner: Naman Mohan Singh

Test Date: [28/03/26]

Test Results: The root StreamBuilder in main.dart listens to authStateChanges() and routes unauthenticated users to AuthScreen. All Firestore queries require authentication; Security Rules deny unauthenticated access.

Additional Comments: NA

2) Requirement: The user should be able to sign in using email and password.

Test Owner: Anubhav Rana

Test Date: [28/03/26]

Test Results: Email and password sign-in is implemented via Firebase Authentication. A password reset mechanism is available via "Forgot password?" which dispatches a Firebase-generated reset link to the registered email.

Additional Comments: NA

3) Requirement: New accounts require email verification before sign-in is permitted.

Test Owner: Karan Kumar

Test Date: [28/03/26]

Test Results: The emailVerified flag is checked after signInWithEmailAndPassword(). If false, the user is immediately signed out, a fresh link is dispatched, and access to HomeScreen is denied.

Additional Comments: NA

4) Requirement: Users can sign in via Google OAuth.

Test Owner: Karan Kumar

Test Date: [28/03/26]

Test Results: Google OAuth is implemented using google_sign_in integrated with Firebase Authentication via GoogleAuthProvider.credential(). A Firestore user document is auto-created for first-time Google users.

Additional Comments: NA

5) Requirement: The user should be able to set and view their profile.

Test Owner: Alankrit Gupta

Test Date: [28/03/26]

Test Results: ProfileScreen reads and displays name, email, photo, and ridesCompleted from Firestore. Users can update their display name and profile photo at any time.

Additional Comments: NA

6) Requirement: Users can sign out.

Test Owner: Bhavishya Gupta

Test Date: [28/03/26]

Test Results: The sign-out action calls FirebaseAuth.instance.signOut(). authStateChanges() emits null, causing navigation to AuthScreen.

Additional Comments: NA

4.1.2 Ride Creation and Management

7) Requirement: Authenticated users can create ride listings with source, destination, and date.

Test Owner: Harshit Gupta

Test Date: [29/03/26]

Test Results: CreateRideScreen presents dropdowns for source and destination (27-item predefined list) and a date picker. Submission writes a Firestore ride document. Source must differ from destination; all fields must be populated.

Additional Comments: NA

8) Requirement: Ride leaders can mark their rides as complete.

Test Owner: Pavi Agarwal

Test Date: [29/03/26]

Test Results: "Mark complete" is visible only to the ride leader on open rides. Tapping updates status to "completed" and increments the leader's ridesCompleted counter via FieldValue.increment(1).

Additional Comments: NA

9) Requirement: Ride leaders can delete their own rides.

Test Owner: Aalok

Test Date: [29/03/26]

Test Results: "Delete" is visible only to the ride leader. A confirmation dialog prevents accidental deletion. On confirmation, the ride document is permanently removed from Firestore.

Additional Comments: NA

4.1.3 Ride Discovery and Filtering

10) Requirement: Users can browse all open rides in real time.

Test Owner: Aryan Nigam

Test Date: [30/03/26]

Test Results: FindRideScreen uses a Firestore StreamBuilder with a .where('status', isEqualTo: 'open') query. The list updates instantly as rides are posted or completed.

Additional Comments: NA

11) Requirement: Users can filter open rides by destination.

Test Owner: Gagandeep Kolambe

Test Date: [30/03/26]

Test Results: The destination dropdown appends a .where('destination', isEqualTo: ...) clause. Only matching rides are returned. "All locations" reverts to the unfiltered query.

Additional Comments: NA

12) Requirement: Users can view ride details including the leader's trust profile.

Test Owner: Naman Mohan Singh

Test Date: [30/03/26]

Test Results: Tapping a ride card opens a bottom sheet that fetches the leader's Firestore document and displays their name and ridesCompleted count alongside ride details.

Additional Comments: NA

4.1.4 Ride Chat

13) Requirement: Users can join any open ride's group chat.

Test Owner: Naman Mohan Singh

Test Date: [31/03/26]

Test Results: "Join and Chat" navigates to ChatScreen with the ride's rideId and rideDestination. Any authenticated user can enter the chat.

Additional Comments: NA

14) Requirement: Chat messages are delivered in real time to all participants.

Test Owner: Naman Mohan Singh

Test Date: [31/03/26]

Test Results: ChatScreen uses a Firestore StreamBuilder on the rides/{rideId}/messages sub-collection ordered by createdAt. Messages appear on all participants' screens immediately.

Additional Comments: NA

15) Requirement: Sending a chat message registers the user as a ride participant.

Test Owner: Anubhav Rana

Test Date: [31/03/26]

Test Results: _sendMessage() calls FieldValue.arrayUnion([currentUser.uid]) on the ride document simultaneously with writing the message. The ride appears in the user's "I Joined" tab on MyRidesScreen.

Additional Comments: NA

4.1.5 My Activity Activity View

16) Requirement: Users can view all rides they are leading.

Test Owner: Naman Mohan Singh

Test Date: [31/03/26]

Test Results: The "I Am Leading" tab queries rides where leaderId == currentUser.uid. All rides posted by the current user appear here regardless of status.

Additional Comments: NA

17) Requirement: Users can view all rides they have joined via chat.

Test Owner: Karan Kumar

Test Date: [31/03/26]

Test Results: The "I Joined" tab queries rides where participants array-contains currentUser.uid. A ride appears after the user sends their first message.

Additional Comments: NA

18) Requirement: Users can open the chat for any ride from My Activity.

Test Owner: Karan Kumar

Test Date: [31/03/26]

Test Results: Each ride card includes an "Open chat" button navigating to ChatScreen with correct rideId and rideDestination. Available in both tabs.

Additional Comments: NA

5 CONCLUSION

- **How effective and exhaustive was the testing?**

Testing was conducted exhaustively across all three phases. Unit tests covered all critical validation paths using fake_cloud_firestore and firebase_auth_mocks, ensuring no dependency on a live database. Every Firestore document schema was verified for correctness. Integration tests walked through all complete user flows and confirmed that frontend actions produced the correct Firestore state. System tests mapped each SRS requirement to a verified test outcome. Edge cases — identical source and destination, empty messages, unverified email, short passwords, and empty fields — were all exercised and confirmed to behave correctly.

- **Which components were not tested adequately?**

Automated testing of non-functional requirements such as performance under high concurrent load, cross-device UI consistency on unusual screen sizes, and long-term Firebase quota behaviour could not be conducted within the project timeline. Firestore Security Rules, while manually reviewed, could benefit from a fuller automated test suite using the @firebase/rules-unit-testing JavaScript library. Push notifications and Firebase Cloud Messaging integration are planned future features and have not yet been tested.

- **What difficulties were encountered during testing?**

The primary challenge was the absence of a custom application server. Because UniPool uses Firebase's client SDK directly from Flutter, there are no HTTP API endpoints to test with tools like Postman or pytest. Adapting the testing strategy to Dart-native mocking libraries required additional setup. Ensuring test isolation — clearing the fake Firestore between tests — required careful tearDown configuration.

- **How could the testing process be improved?**

Testing could be improved by integrating flutter_test-based widget tests into a CI pipeline (GitHub Actions) that runs automatically on every pull request. Firestore Security Rules should be formally tested using @firebase/rules-unit-testing in a separate Node.js test suite run against firebase emulators:start. Firebase Test Lab could be used for automated Robo Tests on a wider range of physical Android devices.

Appendix A – Group Log

Date & Time	Venue	Minutes
20/03/26 11:00 pm	4th floor, Rajeev Motwani Building	Reviewed SRS requirements and mapped them to unit, integration, and system test cases. Set up fake_cloud_firestore and firebase_auth_mocks in pubspec.yaml. Distributed test authorship across group members.
22/03/26 3:00 pm	Online	Wrote unit tests for Authentication (Section 2.1): sign-in valid/invalid, sign-up Firestore document creation, password reset guard, Google sign-in _ensureGoogleProfile(). Resolved firebase_auth_mocks version compatibility. All Auth unit tests passing.
23/03/26 9:00 pm	Online	Wrote unit tests for Ride Handling (2.3): CreateRide document structure, source-equals-destination guard, delete ride, destination filter query. Wrote unit tests for Messaging (2.4): send message and arrayUnion. Wrote Profile unit test (2.2): merge write preservation. All unit tests passing.
24/03/26 9:00 am	1st floor, Rajeev Motwani Building	Manual integration testing sessions for 3.1 (Authentication) and 3.2 (Create Ride). Tested on Android emulator and one physical device. Captured screenshots for all TEST #N entries. Identified and fixed a bug where the date picker allowed past dates.
27/03/26 10:00 pm	1st floor, Rajeev Motwani Building	Manual integration testing for 3.3 (Find & Filter), 3.4 (Edit Profile), 3.5 (Chat). Performed system testing (Section 4) and mapped all 18 requirements to outcomes. All requirements passed.
29/03/26 9:00 pm	1st floor, Rajeev Motwani Building	Finalised Test Document. Verified screenshot placeholder descriptions. Reviewed Dart code snippets for correctness. Completed Conclusion (Section 5) and Group Log. Uploaded to GitHub repository.